«Радар вузов»

CRM контроля взаимодействия ИТ Школы Ростелекома с вузами.
Кейс №6, ЛЦТ 2026. Сопроводительная документация.

Собрано 29.09.2026 командой «make docs-pdf» из документов репозитория: правится исходный Markdown, а не этот файл.
Диаграммы связей, программно-аппаратной архитектуры и базы данных — docs/architecture/DIAGRAMS.md, модель для Archi — docs/architecture/model.xml.

Содержание

1. Архитектура и решения
2. Методы обработки данных
3. Руководство пользователя
4. Руководство администратора
5. Сборка, установка и стенд
6. Меры защиты
7. Нагрузочный тест и масштабирование
8. Диаграммы
9. Соответствие техническому заданию

Архитектура и решения

Версия 0 от 15.09.2026 — черновик на согласование бэкендом и фронтендом. После согласования меняется только PR с одобрением обоих людей; каждое изменение архитектуры — запись в разделе «Решения».

CRM для сотрудников ИТ Школы Ростелекома (кейс №6 ЛЦТ 2026). Ведёт взаимодействия с вузами (B2B, настраиваемый workflow из 14 этапов) и с частными лицами (B2C, свой процесс), сама находит, где процесс застрял (радар), и показывает, какие программы востребованы (рейтинг).

1. Решения

ID	Решение	Почему
ADR-001	Центральная запись — взаимодействие = вуз × ИТ-программа × ИТ-продукт. Вуз — контейнер; договор может покрывать несколько взаимодействий	Совпадает со строкой отчёта из ТЗ; статус передачи и обучение идут по каждому продукту отдельно
ADR-002	Бэкенд: Python 3.12, FastAPI, SQLAlchemy 2 (async, asyncpg), Alembic, PostgreSQL 16+, Redis 7, воркер arq	Разрешено ТЗ; зрелые библиотеки для данных, импорта таблиц и отчётов; асинхронный ввод-вывод под SSE и интеграции
ADR-003	Фронтенд: React 19, Vite, TypeScript, Tailwind 4, shadcn/ui на Radix, TanStack Query	Распространённый стек с типизацией и готовыми доступными компонентами
ADR-004	Контракт contracts/openapi.yaml генерируется из FastAPI командой make openapi; CI сверяет его со схемой приложения; изменения одобряют оба	Один источник правды без ручной синхронизации
ADR-005	История переходов transition только дописывается (триггер БД); этап на любую дату вычисляется из истории	Отчёты за прошлые периоды показывают статусы того времени
ADR-006	Радар и рейтинг детерминированные и объяснимые, без внешних LLM	152-ФЗ, надёжность демо, проверяемость экспертами
ADR-007	Ошибки — application/problem+json с полем code из каталога (раздел 5). Запись вне области видимости — 404 NOT_FOUND	Коды ошибок из ТЗ; не раскрываем существование чужих данных
ADR-008	Пагинация — конверт {items, total, page, page_size} для списков, которые растут вместе с работой: вузы, клиенты, взаимодействия, сигналы, уведомления, выгрузка обмена. Массивом отдаются справочники и настройки (десятки строк, в том числе вендоры и их контакты), списки внутри одной записи (заметки, вложения) и журналы, которые читают с конца: аудит (limit до 500 и offset дальше), запуски синхронизации (50), заявки сайта (200), отчёты (50), очередь обмена и доставки уведомлений (параметр limit), переписка (последние сообщения диалога)	Проще для сгенерированного клиента, чем заголовок X-Total-Count. Конверт у справочника из десятка строк — лишняя обёртка, а журнал запусков листают не дальше последних; граница проходит по тому, может ли список вырасти до тысяч строк, которые пользователю нужно пролистать
ADR-009	Защита от параллельных изменений — поле expected_version в теле перехода; при расхождении 409 INTERACTION_VERSION_CONFLICT	Два пользователя не перезапишут этап друг друга
ADR-010	Поток событий SSE с заголовком Authorization (клиент на fetch), без токена в URL	Токены не попадают в журналы прокси
ADR-011	Локальная разработка без Keycloak: AUTH_MODE=dev и заголовок X-Dev-User; режим запрещён при APP_ENV=production	Фронтенд и тесты работают без поднятого Keycloak
ADR-012	Фронтенд использует keycloak-js, openapi-typescript (типы из контракта), MSW (моки по тому же контракту) и Prettier для генерируемых файлов	Свои реализации этих задач дороже и хуже; типы из контракта не дают интерфейсу разойтись с API
ADR-013	PDF отчёта строит fpdf2 напрямую, без headless-браузера. Шрифт с кириллицей берётся из системы (fonts-dejavu-core в образе)	Chromium с Playwright добавляет около 400 МБ к образу и секунды на старт ради табличного файла. Цена вопроса — диаграммы в PDF рисуются примитивами fpdf2 (ADR-013a), а не ECharts, как в интерфейсе
ADR-013a	Диаграммы статистики в PDF рисует сам fpdf2 прямоугольниками: заголовок, столбики, значения и подписи. Интерфейс те же числа рисует ECharts по полю option, PNG отдаёт он же	ТЗ требует статистику диаграммами в png и pdf. Headless-браузер ради трёх столбчатых диаграмм по-прежнему не нужен (ADR-013), а столбчатая диаграмма — это прямоугольники и подписи; картинку в письмо вузу или на распечатку сервер теперь делает сам
ADR-014	Процесс один на всех: изменения готовятся черновиком и при публикации применяются ко всем открытым записям. Записи с удалённого этапа сами переходят на ближайший предыдущий этап, а если его нет — на следующий; предпросмотр публикации показывает это до подтверждения. Переименование этапа — только администратор. У каждого перехода вперёд есть возврат на шаг назад с комментарием	Уточнения жюри 17.09: версии не должны путать подразделения, изменение процесса не должно останавливать работу, переименование статусов чувствительно, а бюрократический процесс требует возвратов

ID	Решение	Почему
ADR-015	Взаимодействия делятся на группы контрагентов со своим процессом: «Вузы (B2B)» по базовому пути из ТЗ и «Частные лица (B2C)» по своему; администратор добавляет новые группы. Контрагент записи — ровно один: вуз или клиент (client — физическое или юридическое лицо). Продукт необязателен: программы бывают продуктозависимыми и продуктонезависимыми. Расширяет ADR-001	Уточнения жюри 17.09: B2C «прямо в цель» — обучение частных и юридических лиц идёт по своему workflow; процесс меняется для группы целиком; программа управления проектами идёт с «Ягой», а часть программ — без продукта
ADR-016	Уведомления: лента в интерфейсе и доставка в Telegram, Max и почту (SMTP) за общим интерфейсом отправителя. Доставка асинхронная через очередь notification_delivery с повторами (1, 5, 15, 60 минут, до пяти попыток); секрет канала — только имя переменной NOTIFY_*. Эскалация зависших записей идёт роли (руководитель команды КАМа или администраторы), а не человеку; срок — настройка stalled_escalation	Уточнения жюри 17.09: уведомления о смене этапов в Telegram, Max и подключаемую почту «хотя бы на заглушках»; зависшая заявка — уведомление вышестоящей роли через настраиваемый срок. Очередь не даёт медленному мессенджеру задерживать переход, а префикс переменных не позволяет через настройку канала вывести наружу чужой секрет
ADR-017	Файлы вложений и отчётов лежат в S3-совместимом хранилище: на стенде в Yandex Cloud — Yandex Object Storage, в закрытом контуре и в локальном compose — MinIO или любое другое S3. Код работает через интерфейс Storage; STORAGE_BACKEND=local оставляет каталог на диске для разработки без контейнеров. Бакет создаётся при первом обращении, состояние хранилища входит в /api/health	Жюри назвало S3-хранилище (например MinIO) частью минимальной архитектуры и раздельное хранение файлов и реляционных данных — правильным подходом. Клиент minio работает с любым S3-совместимым сервисом и типизирован. Стенд в Yandex Cloud хранит файлы в Yandex Object Storage (infra/docker-compose.yandex.yml): образ MinIO к осени 2026 перестал скачиваться из публичных реестров, а бакет с версионированием сам даёт надёжность и копию; свой MinIO остаётся для локальной разработки
ADR-018	Обмен с LMS и CMS сайта двусторонний. Входящий — раз в час (выключается у источника флагом pull_enabled). Исходящий — transactional outbox: изменение записи в той же транзакции ставит отметку для каждого получателя, воркер раз в минуту строит документы radar-vuzov/interaction@1 по последнему состоянию и отправляет пакетом JSON; отказ получателя повторяется с растущей паузой без предела попыток, пока получатель не примет пакет (флаг push_enabled выключает отправку). Токен источника — из переменной окружения, имя которой хранит secret_ref. Тот же документ отдаёт API выгрузки (updated_since — забирать изменения по расписанию, окно перекрывается на пять минут)	Жюри: CRM — ядро процесса с двусторонней интеграцией, обмен JSON по API синхронно или по расписанию, онлайн не нужен; в выгрузке статус, ответственный, вуз, программа, продукт и ключи связей с файлами в S3 и записями БД. Outbox не теряет изменения при отказе получателя и не шлёт ничего по откатившейся транзакции; контракт приёма описан нами, под контракт заказчика меняется только адаптер
ADR-019	Импорт принимает CSV наравне с xls и xlsx: кодировка определяется по BOM, строгому UTF-8, иначе выбирается из cp1251, KOI8-R и cp866 по доле строчной кириллицы; разделитель — по образцу строк и подсказке Excel sep=; кодировку можно указать вручную. Отчёты выгружаются и в CSV — UTF-8 с BOM или cp1251. В табличных выгрузках значения, начинающиеся с =, +, -, @, экранируются апострофом	Жюри назвало кодировки «мелочью, на которой сыпятся»: файл должен загружаться и выгружаться без слетевшего формата. Своё определение из двух десятков строк проще и предсказуемее общей библиотеки для узкой задачи «русский текст в одной из четырёх кодировок»; апостроф закрывает CSV-инъекцию, когда выгрузку открывают в Excel
ADR-020	Состояние записи (active, paused, completed, cancelled) отделено от этапа процесса и меняется методом PUT /interactions/{id}/status с причиной для паузы и отмены. Приостановленная запись не копит сигналы простоя и не участвует в переходах; отменённая уходит из рабочих списков, но остаётся в карточке, истории и отчёте с фильтром по состоянию; завершённую и отменённую можно вернуть в работу, если связка «вуз — программа — продукт» ещё свободна	Пауза и отмена — обычная жизнь сделки: вуз взял паузу до бюджета, заявка оказалась ошибочной. Без отдельного состояния такие записи либо висят просроченными в радаре, либо удаляются вместе с историей, а история по ТЗ только дописывается
ADR-021	Порядок продвижения курсов задаётся вручную: program.priority от 0 до 100, справочник и рейтинг (order=priority) показывают отмеченные курсы первыми. Место и балл рейтинга при этом считаются только по данным и не меняются; приоритет правят руководитель и администратор, изменение пишется в аудит	Уточнение жюри 17.09: «приоритеты курсов можно и вручную». Данных LMS и сайта не хватает на решения вроде «в этом году продвигаем ИИ», но подменять ими расчётный рейтинг нельзя — иначе рейтинг перестаёт быть объяснимым (ADR-006)

ID	Решение	Почему
ADR-022	Кэш двухуровневый. На сервере в Redis лежат готовые ответы справочников и рейтинга: ключ содержит номер поколения, изменение справочника увеличивает номер, и прежние ключи просто перестают попадаться. У клиента — условные запросы: ответы справочников, рейтинга и карточки идут с ETag и Cache-Control: private, no-cache, файлы вложений и готовых отчётов — с ETag и тем же private, no-cache (содержимое не меняется, но доступ к нему может пропасть, поэтому браузер переспрашивает сервер и получает короткий 304); immutable — только у публичных картинок справки; повтор с If-None-Match отвечает 304 без тела. Недоступный Redis означает «в кэше ничего нет», а не ошибку	FR-13 «кэш действий пользователя»: повторное открытие карточки и списков не должно стоить ни запроса к базе, ни лишнего трафика. private и обязательная проверка на сервере не дают показать данные, которые пользователь больше не вправе видеть, а хэш файла — готовый и точный ETag
ADR-023	Обучающиеся и преподаватели — список у записи (participant), а не отдельный справочник людей. ФИО видит тот, кто видит запись; почта хранится зашифрованной, в списке показана сокращённо, а целиком отдаётся отдельным методом с записью в аудит. Список ведётся руками и файлом (JSON, CSV, XLSX, XLS), человек находится по HMAC-отпечатку почты, а без почты — по ФИО и роли (частный уникальный индекс по ключу ФИО). Через PARTICIPANT_RETENTION_DAYS (180) дней после завершения или отмены записи ФИО и почта стираются ночным заданием, строка остаётся для счёта	Уточнение жюри 17.09: нужны списки обучающихся и преподавателей с ФИО и email. Общий справочник людей — это отдельная база персональных данных без владельца; список у записи живёт ровно столько, сколько идёт обучение, и удаляется вместе с ним. Отпечаток нужен потому, что Fernet недетерминирован: искать по зашифрованному полю нельзя (152-ФЗ, ADR-010)
ADR-024	Переписка сотрудников живёт внутри сервиса: таблица message, сообщение видят только отправитель и получатель, ссылка на запись показывается собеседнику, только если запись в его области видимости. Новое сообщение приходит событием message.created адресату; во внешние каналы переписка не уходит	Уточнение жюри 17.09: мессенджер внутри сервиса «хотя бы заглушкой». Обсуждение сделки уходит в личные чаты и теряется вместе с контекстом; здесь разговор остаётся рядом с записью, а область видимости работает так же, как в списках (ADR-007). Пересылать переписку в Telegram нельзя: это разговор людей, а не событие системы
ADR-025	Руководство пользователя живёт в коде: разделы — Markdown в app/help/, API отдаёт их по роли (GET /api/v1/help), а make help собирает из них docs/USER_GUIDE.md; CI сверяет документ со справкой так же, как контракт со схемой	Жюри просило руководства «лучше встроенные в продукт». Два текста — в интерфейсе и в репозитории — разошлись бы за неделю, поэтому источник один, а документ собирается. Разделы про администрирование не показываются КАМу: справка не должна рассказывать о том, чего человек не может
ADR-026	Оплаты частных лиц приходят выгрузкой платёжной системы (POST /api/v1/payments/import, JSON как у кейсодержателя, null среди записей — ошибка строки): человек находится по HMAC-отпечатку почты (client.email_fp) или заводится карточкой, его запись B2C по курсу находится или заводится и переходит на «Зачисление в LMS» с номером заявки и потока в истории; подтверждением оплаты служит сама выгрузка, документ к переходу не прикладывается. Номер заявки уникален (payment.order_no). Для зачисления сервер отдаёт XLSX «Загрузка пользователей» по шаблону LMS дословно: 30 колонок, второй лист со списками и выпадающие списки в колонках «Пол» и «Образование»; CRM заполняет ФИО, телефон и почту, паспорт и СНИЛС не хранит. Контакты вендоров — отдельная таблица vendor_contact с шифрованием и связью с продуктами; таблица кейсодержателя «Вендоры» грузится импортом справочников (vendor-contacts) как есть	Кейсодержатель прислал выгрузку оплат, шаблон загрузки пользователей в LMS и таблицу вендоров: оплата — событие, после которого запись B2C идёт на зачисление, и проводить её руками по каждому человеку незачем. Паспорт и СНИЛС CRM для работы не нужны, а хранить лишние персональные данные нельзя (152-ФЗ) — их человек вносит в LMS сам. Заголовки шаблона повторены с опечатками оригинала: загрузчик LMS сверяет колонки по тексту

2. Компоненты

Три диаграммы — связей, программно-аппаратной архитектуры и базы данных — лежат в docs/architecture/DIAGRAMS.md; полная модель для Archi — docs/architecture/model.xml.

Сервис	Технология	Назначение
web	nginx	Отдаёт SPA, проксирует /api и /auth, TLS на стенде
api	FastAPI + uvicorn	REST /api/v1, SSE /api/v1/events, Swagger UI /api/docs
worker	arq	Отчёты, импорт, синхронизация LMS и сайта, пересчёт радара и рейтинга, подсказки норм
postgres	PostgreSQL 16+	Доменные данные, история, витрины

Сервис	Технология	Назначение
S3-хранилище	Yandex Object Storage на стенде, MinIO в локальном compose и закрытом контуре	Файлы вложений и отчётов отдельно от базы; код одинаковый, меняются только адрес и ключи
redis	Redis 7	Очередь, кэш готовых ответов справочников и рейтинга (ADR-022), поток событий
keycloak	Keycloak 26+	Вход OIDC, роли, политики паролей и сессий
mock-lms, mock-site	FastAPI	Данные LMS и заявки сайта по контракту — на демо-стенде вместо систем заказчика
mock-messengers	FastAPI	Заглушка Telegram Bot API и Bot API Max: принимает уведомления и показывает полученное
mailpit	Mailpit	Почтовый сервер для писем-уведомлений на стенде, просмотр писем в браузере

Вложения и отчёты хранятся через слой Storage: S3-совместимое хранилище (на стенде — Yandex Object Storage, локально и в закрытом контуре — MinIO), каталог на диске — для разработки без контейнеров (ADR-017).

Структура бэкенда

```
backend/
├── app/
│   ├── main.py          # сборка приложения, роутеры, обработчики ошибок
│   ├── worker.py        # воркер arq: ночные задачи, отчёты, обмен, уведомления
│   ├── help/            # встроенная справка (Markdown и скриншоты)
│   ├── core/            # config, db, errors, security, scope, cache, events, storage, crypto
│   └── modules/
│       ├── catalogs/    # вузы, направления, программы, продукты, пользователи, группы
│       ├── clients/     # клиенты вне вузов: люди и организации
│       ├── vendors/     # вендоры и их контакты
│       ├── workflow/    # шаблоны, версии, этапы, правила, нормы, редактор процесса
│       ├── interactions/  # взаимодействия, договоры, переходы, заметки, состояние
│       ├── attachments/   # документы этапов
│       ├── participants/  # обучающиеся и преподаватели записи
│       ├── payments/    # оплаты частных лиц и файл зачисления в LMS
│       ├── radar/       # сигналы
│       ├── analytics/   # рейтинг и статистика
│       ├── reports/     # отчёты в фоне
│       ├── imports/     # загрузка выгрузок заказчика
│       ├── integrations/  # LMS и сайт: входящий обмен и очередь исходящего
│       ├── exchange/    # документ обмена radar-vuzov/interaction@1
│       ├── notifications/ # лента, каналы, эскалация
│       ├── messages/    # переписка сотрудников
│       ├── events/      # поток событий SSE
│       ├── views/       # сохранённые виды списков
│       ├── metrics/     # месячные показатели LMS и сайта
│       ├── admin/       # пользователи, правила доступа, настройки, справочники файлом
│       ├── audit/       # журнал аудита
│       └── help/        # отдача справки по роли
├── migrations/          # Alembic
├── mocks/               # заглушки LMS, сайта и мессенджеров
├── scripts/             # экспорт OpenAPI, справки и PDF документации, демо-данные
└── tests/
```

Слои модуля: router.py (HTTP: разбор запроса и вызов сервиса) → service.py (правила, область видимости, транзакции и запросы) → models.py / schemas.py. Роутеры не обращаются к БД напрямую.

Ключевые потоки

1. Смена этапа (p95 ≤ 1 с). SPA сразу показывает новый этап → POST /api/v1/interactions/{id}/transitions с expected_version → сервис проверяет область видимости и правило перехода → одна транзакция: запись в transition и обновление interaction → пересчёт сигналов этого взаимодействия → событие в Redis → SSE владельцу и руководителю.
2. Отчёт. POST /api/v1/reports → 202 и id задания → воркер строит xlsx (openpyxl), xls (xlwt), csv, json или pdf (fpdf2, ADR-013) → прогресс по SSE → скачивание.
3. Импорт xls, xlsx и csv. Загрузка (кодировка CSV определяется или задаётся) → маппинг колонок →

предпросмотр → применение: каталоги, договоры и взаимодействия; даты подписания и срока лицензии пишутся в историю → пересчёт радара.
4. Интеграции, входящие. Воркер по расписанию получает JSON из LMS (Moodle Web Services, СЭО 3KL) и сайта → месячные метрики → заявка сопоставляется с взаимодействием по (вуз, программа, продукт) или создаёт новое на первом этапе.
5. Интеграции, исходящие. Изменение записи (создание, переход, смена ответственного, документ, импорт, заявка с сайта, изменение процесса) в своей транзакции пишет отметку в integration_outbox → воркер раз в минуту строит документы radar-vuzov/interaction@1 и отправляет их пакетом в LMS и CMS → принятые помечаются отправленными, отклонённые по содержанию ждут человека, при недоступности получателя — повтор через 1, 5, 15, 60 минут.

3. Модель данных (PostgreSQL)

Конвенции: PK id uuid; created_at, updated_at — timestamptz в UTC; у каталогов мягкое удаление archived_at; перечисления — text + CHECK; внешние ключи по умолчанию ON DELETE RESTRICT — запись с историей не удаляется каскадом. Исключения выбраны явно: CASCADE у подчинённых строк, которые без владельца не имеют смысла (связи program_product и product_direction, строки импорта, адреса и доставки уведомлений, личные виды, переписка, очередь обмена, список участников записи), SET NULL у ссылок «кто создал» и необязательных связей (created_by, updated_by, team_id, manager_user_id, interaction_id у заявки сайта и сообщения). База создаётся с локалью UTF-8 — в локали C поиск без учёта регистра не работает для кириллицы. Нотация: поле → таблица — внешний ключ.

Каталоги

```
counterparty_group id, code UNIQUE, name UNIQUE, description, workflow_template_id → workflow_template,
                position, archived_at                -- вузы (B2B), частные лица (B2C) и новые группы
client           id, kind (person|organization), name, inn UNIQUE WHERE inn IS NOT NULL, city,
                email_enc, email_fp (HMAC почты), phone_enc, created_by → app_user, archived_at  -- ПДн
university       id, name UNIQUE, short_name, region, city, is_priority2030, archived_at
direction        id, code UNIQUE, name
program          id, direction_id → direction, name, lms_course_ref, priority (0–100, ручной)
vendor           id, name UNIQUE
product          id, vendor_id → vendor, name
product_direction PK(product_id, direction_id)
program_product   PK(program_id, product_id), is_default,
                UNIQUE(program_id) WHERE is_default, UNIQUE(product_id) WHERE is_default
                -- основная пара: программа продукта при импорте, продукт программы у заявки сайта;
                -- единственная связь программы (продукта) считается основной и без флага
contact_person    id, university_id → university, full_name, position, email_enc, phone_enc   -- ПДн
vendor_contact    id, vendor_id → vendor, full_name, name_key, email_enc, phone_enc, channels text[]
                (email|telegram|phone), archived_at, UNIQUE(vendor_id, name_key) WHERE archived_at IS NULL
vendor_contact_product PK(contact_id, product_id)                    -- за какие продукты отвечает
team             id, name, manager_user_id → app_user
app_user         id, keycloak_sub UNIQUE, email UNIQUE, full_name, role (kam|manager|admin), team_id → team, is_active
```

Workflow

```
workflow_template     id, name, is_default, archived_at
workflow_version      id, template_id → workflow_template, version_no, status (draft|published|retired), published_at
stage                 id, version_id → workflow_version, code, name, position, kind (start|normal|final),
                bulk_allowed, required_document_types text[]
stage_transition_rule id, version_id, from_stage_id → stage, to_stage_id → stage, requires_comment, requires_attachment
stage_norm            id, template_id, stage_code, norm_days, source (manual|suggested),
                suggested_median_days, suggested_percentile_days, sample_size,
                UNIQUE(template_id, stage_code)
```

Базовый шаблон, 14 этапов:

№	Код	Название
1	contact_search	Поиск контактов
2	communication	Коммуникация
3	meeting	Встреча
4	documents_exchange	Обмен документами
5	documents_revision	Доработка документов (необязательный)

№	Код	Название
6	signing	Подписание
7	materials_transfer	Передача материалов
8	implementation_support	Поддержка внедрения
9	teacher_training	Обучение преподавателей
10	curriculum_update	Обновление программы
11	classes	Ведение занятий
12	docs_update	Обновление документации
13	teacher_upskilling	Повышение квалификации
14	stage_control	Контроль этапов (финальный)

• Переход по умолчанию — на следующий этап; с «Обмена документами» можно сразу на «Подписание», минуя доработку. У каждого перехода вперёд есть возврат на шаг назад: комментарий обязателен, документ — нет. Комментарий обязателен для каждого перехода.
• Обязательные документы: «Подписание» — подписанный договор, «Передача материалов» — акт передачи, «Обучение преподавателей» — подтверждение обучения. Требование действует на выход вперёд; возврат назад документа не требует.
• Процесс у группы один (ADR-014). Переименование этапа применяется сразу и доступно только администратору. Добавление, удаление и перестановка этапов готовятся черновиком; при публикации все открытые записи переходят на новую схему. Запись с удалённого этапа переходит на ближайший предыдущий этап прежней схемы, который остался, а если такого нет — на ближайший следующий; migration_map задаёт этап явно. Если дошли до другого этапа, дни на этапе считаются заново. Черновик, который переименовывает этапы, публикует только администратор.

Взаимодействия

```
contract          id, university_id → university, number, signed_at, license_signed_at, license_valid_until,
                  license_term_years, transfer_status, UNIQUE(university_id, number)
interaction       id, group_id → counterparty_group, university_id | client_id (ровно один), program_id,
                  product_id (может быть пуст), contract_id → contract, workflow_version_id,
                  current_stage_id → stage, stage_entered_at, owner_user_id → app_user,
                  status (active|paused|completed|cancelled), source (manual|import|site|lms|demo),
                  version, last_activity_at,
                  UNIQUE NULLS NOT DISTINCT (university_id, program_id, product_id) WHERE status <> 'cancelled',
                  UNIQUE NULLS NOT DISTINCT (client_id, program_id, product_id) WHERE status <> 'cancelled'
interaction_contact PK(interaction_id, contact_person_id), role
transition        id, interaction_id, from_stage_id, to_stage_id, occurred_at, actor_user_id, comment,
                  source (manual|bulk|import|integration|migration)       -- только INSERT
interaction_note  id, interaction_id, author_user_id, text
attachment        id, interaction_id, transition_id, stage_id, document_type, file_name, mime_type,
                  size_bytes, sha256, storage_key, uploaded_by
assignment_change id, interaction_id, from_user_id, to_user_id, changed_by, changed_at, reason
```

• Группа определяет процесс: запись встаёт на первый этап действующей схемы процесса своей группы, нормы и радар берутся из того же шаблона. Процесс группы с открытыми записями не заменяется — его меняют в редакторе, и записи переходят на новую схему (ADR-014).
• Процесс «Частные лица (B2C)»: заявка → консультация → договор-оферта → оплата → зачисление в LMS → обучение → итоговая аттестация → завершено. Выход с оферты, оплаты и аттестации требует документа (подписанная оферта, подтверждение оплаты, сертификат).
• Статус взаимодействия: рабочее состояние — active. completed и paused ставит человек через администрирование; переход на финальный этап сам по себе ничего не завершает.

Импорт, интеграции, аналитика, аудит

```
import_profile    id, name, file_kind (xls|xlsx|csv), column_map jsonb
import_batch      id, profile_id, file_name, file_kind, encoding, delimiter, status (uploaded|previewed|applied|failed),
                  headers jsonb, column_map jsonb, stats jsonb, uploaded_by, applied_at
import_row        id, batch_id, row_no, raw jsonb, resolution (new|update|conflict|skip|needs_program),
                  detail  -- причина, по которой строка требует внимания
integration_source id, kind (lms|site), base_url, secret_ref, is_mock, schedule_cron, last_sync_at,
                  pull_enabled, push_enabled, last_push_at
sync_run          id, source_id, direction (pull|push), started_at, finished_at, status, stats jsonb, error_code
integration_outbox id, source_id, interaction_id, reason, status (pending|sent|failed), change_seq, locked_until,
```

```
                attempts, next_attempt_at, last_error, sent_at,
                UNIQUE(source_id, interaction_id) WHERE status = 'pending'
site_application   id, external_id UNIQUE, university_id, program_id, university_name, program_name,
                contact_name (ПДн заявителя), comment, received_at, match_status (matched|unmatched),
                interaction_id → interaction (SET NULL)
payment           id, order_no UNIQUE, interaction_id → interaction, client_id → client, program_id,
                stream_no, imported_by    -- оплата из платёжной системы, без ПДн
participant       id, interaction_id → interaction, role (student|teacher), full_name, name_key (ФИО без
                регистра и знаков), email_enc, email_fp (HMAC для поиска дублей), external_ref,
                source (manual|import|lms), created_by, archived_at,
                UNIQUE(interaction_id, email_fp) WHERE email_fp IS NOT NULL,
                UNIQUE(interaction_id, role, name_key) WHERE email_fp IS NULL AND archived_at IS NULL
program_metric    id, university_id, program_id, period_month, metric (applications|students|streams), value, source
radar_signal      id, interaction_id, kind, severity (low|medium|high), detected_at, resolved_at, evidence jsonb,
                UNIQUE(interaction_id, kind) WHERE resolved_at IS NULL
rating_weight_set  id, name, w_applications, w_students, w_streams, is_default   -- сумма весов 100
report_job        id, requested_by, params jsonb, status, progress, file_key, error_code
message           id, sender_id → app_user, recipient_id → app_user, interaction_id (может быть пуст),
                body, created_at, read_at
saved_view        id, user_id → app_user, page (interactions|radar|reports|rating|clients), name,
                filters jsonb, columns jsonb, UNIQUE(user_id, page, name)
data_access_rule   id, subject_user_id | subject_role, effect (allow|deny), scope_kind (university|direction|program|group), scope_id
audit_log         id bigserial, occurred_at, actor_user_id, action, entity_kind, entity_id, before, after, trace_id  -- только INSERT
app_setting       key, value jsonb                  -- radar_thresholds, stalled_escalation
notification      id, user_id → app_user, kind (stalled_interaction|stage_changed|workflow_changed|channel_test),
                title, body, interaction_id, payload jsonb, dedupe_key UNIQUE, created_at, read_at
notification_channel  kind PK (telegram|max|email), name, is_enabled, is_mock, settings jsonb, secret_ref, kinds text[]
notification_address  PK(user_id, channel_kind), address, is_enabled    -- чат, пользователь Max, email
notification_delivery id, notification_id, channel_kind, status (pending|sent|failed), attempts,
                next_attempt_at, locked_until, last_error, sent_at
```

Поля xlsx заказчика → модель: «Название ВУЗа» → university; «Вендор», «ПО» → vendor, product; «Номер договора», «Подписание лицензии», «Срок действия лицензии» → contract; «Статус по передаче» → contract.transfer_status и этап по карте значений; «ФИО Менеджера» → interaction.owner_user_id; «Ответственные от ВУЗа» → contact_person; «Комментарий» → interaction_note.

4. Доступ и роли

Роль	Видит взаимодействия	Может сверх просмотра
kam — КАМ	свои (owner_user_id = пользователь)	переходы, комментарии, файлы в своих взаимодействиях
manager — руководитель	своей команды и свои	назначать и снимать ответственных, веса рейтинга, импорт, изменение workflow без переименования этапов
admin — администратор	все	пользователи, правила доступа к данным, интеграции, настройки, аудит, переименование этапов

- Keycloak: realm radar-vuzov, публичный клиент radar-web (Authorization Code + PKCE), аудитория API radar-api, роли в claim realm_access.roles.
- При первом входе пользователь создаётся в app_user по sub и email из токена; роль берётся из токена, команда — из БД.
- Область видимости применяет сервисный слой ко всем запросам; запись вне области — 404 NOT_FOUND.
- Правила доступа администратора адресуют вуз, направление, программу или группу контрагентов: так команда B2B не видит частных лиц, а команда B2C — вузы.

5. API

- Префикс /api/v1, JSON, даты ISO 8601 в UTC. Swagger UI — /api/docs, схема — /api/openapi.json, копия — contracts/openapi.yaml.
- Списки: ?page=1&page_size=50 (максимум 200) → {items, total, page, page_size}. Одинаковые фильтры у списков, отчётов и JSON-выгрузки: period_from, period_to, university_id, direction_id, program_id, product_id, owner_id, stage_code, search (повторяемые параметры для множественного выбора). Взаимодействие попадает в период, если создано не позже period_to и либо всё ещё активно, либо в периоде был хотя бы один переход. Даты сравниваются в UTC.
- Авторизация: Authorization: Bearer <access_token>.

Методы

Метод	Назначение
GET /api/health	Состояние сервиса и БД
GET /api/v1/me	Профиль: id, ФИО, роль, команда, область видимости
GET /api/v1/help, GET /api/v1/help/{slug}	Встроенное руководство: разделы по роли и текст раздела в Markdown
GET /api/v1/help/images/{name}	Скриншот из справки. Без авторизации: <img> не передаёт токен, а на картинках только демо-данные; имя файла проверяется по белому списку
GET /api/v1/universities, /{id}	Вузы со счётчиками взаимодействий и открытых сигналов
GET /api/v1/directions, /programs, /products, /users	Справочники
PUT /api/v1/programs/{id}/priority	Ручной приоритет курса (0–100); меняют руководитель и админ
GET /api/v1/counterparty-groups	Группы контрагентов и их процессы
GET, POST /api/v1/saved-views, PATCH, DELETE /api/v1/saved-views/{id}	Свои виды списка: фильтры и колонки под своим названием
GET /api/v1/messages, GET /api/v1/messages/{peer_id}, POST /api/v1/messages, POST /api/v1/messages/{peer_id}/read	Переписка с коллегой: список диалогов с непрочитанными, сама переписка, отправка, отметка о прочтении
GET, POST /api/v1/clients, GET /api/v1/clients/{id}	Клиенты вне вузов; организации видны всем, люди — тем, кто с ними работает; просмотр карточки человека пишется в аудит
GET /api/v1/workflows	Процессы: действующая схема, черновик изменений, группы
GET /api/v1/workflows/{template_id}, /norms, PUT /norms/{stage_code}, POST /norms/{stage_code}/accept-suggestion	Схема и нормы процесса любой группы
GET /api/v1/workflows/default	Опубликованная версия базового workflow с этапами и правилами
POST /api/v1/workflows	Новый шаблон процесса
POST /api/v1/workflows/{template_id}/versions	Черновик версии копией последней
PATCH /api/v1/workflow-versions/{id}	Этапы, порядок, нормы и правила черновика
PATCH /api/v1/stages/{id}	Переименование этапа в действующей схеме — только администратор
POST /api/v1/workflow-versions/{id}/publish-preview	Предпросмотр публикации для окна подтверждения: переименования, удалённые этапы, куда и сколько записей переедет
POST /api/v1/workflow-versions/{id}/publish	Публикация: все открытые записи переходят на новую схему, с удалённых этапов — на соседний
GET /api/v1/interactions	Взаимодействия с фильтрами: группа, клиент, вуз, программа, продукт, КАМ, этап, дни на этапе, версия, открытые сигналы
POST /api/v1/interactions	Запись вручную: группа, вуз или клиент, программа, продукт, ответственный; встаёт на первый этап процесса группы
GET /api/v1/interactions/{id}	Карточка: договор, история, допустимые переходы с требованиями, сигналы
POST /api/v1/interactions/{id}/transitions	Переход: to_stage_id, comment, expected_version, attachment_ids
PUT /api/v1/interactions/{id}/status	Состояние записи: status, reason, expected_version — пауза, завершение, отмена, возврат в работу
GET, POST /api/v1/interactions/{id}/notes	Заметки по записи; заметка снимает сигнал о простое
POST /api/v1/interactions/bulk-transitions	Групповой переход: interaction_ids, to_stage_code, comment
PUT /api/v1/interactions/{id}/owner	Смена ответственного: owner_id, reason, expected_version
POST /api/v1/interactions/bulk-owner	Групповая передача записей другому КАМу
POST /api/v1/interactions/{id}/attachments	Загрузка документа (multipart: file, document_type)
GET /api/v1/interactions/{id}/attachments	Документы взаимодействия
GET, POST /api/v1/interactions/{id}/participants	Обучающиеся и преподаватели записи; почта в списке сокращена
GET /api/v1/participants/{id}/contact	Почта участника целиком; просмотр пишется в аудит
DELETE /api/v1/participants/{id}	Убрать участника: строка уходит, почта стирается
POST /api/v1/interactions/{id}/participants/import	Список файлом с предпросмотром
POST /api/v1/payments/import	Оплаты частных лиц из платёжной системы: клиент и запись B2C находятся или заводятся, запись переходит на «Зачисление в LMS»; предпросмотр (dry_run); руководитель и админ
GET /api/v1/payments/lms-users	XLSX «Загрузка пользователей» по шаблону LMS для записей на этапе зачисления, по курсу и потоку; пишется в аудит
GET /api/v1/vendors	Вендоры с id, продуктами и числом контактов
GET, POST /api/v1/vendors/{id}/contacts	Контакты вендора; просмотр пишется в аудит, добавляют руководитель и админ
POST /api/v1/vendor-contacts/{id}/archive	Контакт вендора в архив: почта и телефон стираются
GET /api/v1/interactions/{id}/participants/export	Список файлом: JSON, CSV или XLSX
GET /api/v1/attachments/{id}/file	Файл вложения с исходным именем
POST /api/v1/imports	Загрузка выгрузки xls, xlsx или csv: колонки, подсказка соответствия, определённые кодировка и разделитель; encoding — указать вручную

Метод	Назначение
PUT /api/v1/imports/{id}/mapping	Соответствие колонок и предпросмотр по строкам
POST /api/v1/imports/{id}/apply	Применение загрузки
GET /api/v1/import-profiles	Сохранённые соответствия колонок
GET /api/v1/signals	Открытые сигналы радара с доказательствами
GET /api/v1/signals/summary	Матрица «КАМ × вид сигнала» для тепловой карты
POST /api/v1/reports	Заказать отчёт: период, фильтры списка, колонки, формат xlsx, xls, csv (UTF-8 с BOM или windows-1251), pdf или json
GET /api/v1/reports, /{id}	Свои задания и состояние с прогрессом
GET /api/v1/reports/{id}/file	Готовый файл отчёта
GET /api/v1/integrations	Источники LMS и сайта
POST /api/v1/integrations/{id}/sync	Ручная синхронизация источника
GET /api/v1/integrations/{id}/runs	Журнал обмена в обе стороны (direction: pull, push) с кодами ошибок
POST /api/v1/integrations/{id}/push, GET /{id}/outbox	Отправить изменения сейчас; очередь отправки получателю
PATCH /api/v1/integrations/{id}	Включить или выключить отправку изменений источнику (администратор)
GET /api/v1/interactions/{id}/export	Документ обмена radar-vuzov/interaction@1 по записи
GET /api/v1/exchange/interactions	Выгрузка документов обмена, updated_since — только изменённые; окно перекрывается на пять минут назад, чтобы не терять изменения, зафиксированные во время опроса
GET /api/v1/site-applications	Заявки с сайта; match_status=unmatched — очередь на разбор (видна всем руководителям), сопоставленные — только в области видимости их записи
POST /api/v1/site-applications/{id}/match	Привязать заявку к взаимодействию; чужую сопоставленную заявку перепривязать нельзя
GET /api/v1/analytics/rating	Рейтинг: место, балл, вклад каждой метрики, полнота данных, ручной приоритет; order=priority — порядок продвижения
GET, PUT /api/v1/analytics/rating/weights	Веса по умолчанию; меняют руководитель и админ
GET /api/v1/analytics/stats/funnel, /stage-durations, /distribution	Статистика с настройками ECharts
GET /api/v1/analytics/stats/report	Те же три диаграммы в PDF: столбики рисует сервер, браузер не нужен
GET, PATCH /api/v1/admin/users	Сотрудники: роль, команда, доступ
GET, POST /api/v1/admin/teams	Команды
GET, POST, DELETE /api/v1/admin/access-rules	Правила доступа к данным
GET, PUT /api/v1/admin/settings	Настройки приложения
GET /api/v1/admin/audit	Журнал аудита, новые сверху: limit записей со сдвигом offset; фильтры по действию, виду и id объекта, сотруднику и периоду
POST /api/v1/admin/catalogs/{kind}, /{id}/archive	Каталоги: добавление и архивирование
POST /api/v1/admin/catalogs/{kind}/import	Справочник файлом JSON, CSV, XLSX или XLS: предпросмотр (dry_run) и применение с итогом по каждой строке
GET /api/v1/admin/catalogs/{kind}/export	Выгрузка справочника в JSON, CSV или XLSX в том виде, в каком его принимает загрузка
POST, PATCH /api/v1/admin/counterparty-groups, POST /{id}/archive	Группы контрагентов: добавление, процесс группы, архив без открытых записей
GET, POST /api/v1/universities/{id}/contacts	Контакты вуза; просмотр пишется в аудит
POST /api/v1/contacts/{id}/archive	Убрать контакт в архив: почта и телефон стираются, строка остаётся для истории
GET /api/v1/events	Поток событий text/event-stream
GET /api/v1/notifications, POST /{id}/read, POST /read-all	Лента уведомлений сотрудника
GET, PUT /api/v1/me/notification-addresses/{channel_kind}	Свой адрес в Telegram, Max или почте
GET, PATCH /api/v1/admin/notification-channels/{kind}, POST /{kind}/test	Каналы уведомлений и пробное сообщение
GET /api/v1/admin/notification-deliveries	Журнал доставки
POST /api/v1/admin/escalations/run	Проверить зависшие записи сейчас, не дожидаясь ночи
GET /api/v1/workflows/default/norms	Нормы этапов с подсказками по истории
PUT /api/v1/workflows/default/norms/{stage_code}	Норма вручную
POST /api/v1/workflows/default/norms/{stage_code}/accept-suggestion	Принять подсказку

Вложения лежат за интерфейсом Storage (app/core/storage.py): на стенде это бакет Yandex Object Storage, в локальном compose — MinIO (в обоих случаях STORAGE_BACKEND=s3), без контейнеров — каталог UPLOAD_DIR; ключ одинаковый: interactions/{interaction_id}/{attachment_id}. Отсутствующий объект S3 выглядит для сервисов как отсутствующий файл и превращается в NOT_FOUND. Тип файла проверяется дважды — по расширению и по сигнатуре содержимого, поэтому exe с именем .pdf отклоняется кодом FILE_TYPE_NOT_ALLOWED; предел размера задаёт MAX_UPLOAD_MB (по ТЗ 25 МБ), превышение — FILE_TOO_LARGE. Документ привязывается к этапу, на котором загружен, и закрывает сигнал «нет документа»;

выход с этапов «Подписание», «Передача материалов» и «Обучение преподавателей» без документа недоступен.

Интеграции спрятаны за интерфейсами LmsClient, SiteClient и InteractionReceiver (app/modules/integrations/clients.py), поэтому код не зависит от того, отвечает ли настоящая Moodle или мок из backend/mocks/. Обмен двусторонний (ADR-018): очередь захватывается арендой (locked_until), а не блокировкой на время сетевого вызова, и отправка засчитывается, только если счётчик изменений записи change_seq не поменялся — изменение, сделанное во время отправки, уйдёт следующим пакетом. Изменения записей CRM уходят обоим получателям документом radar-vuzov/interaction@1 (app/modules/exchange/) — статус и этап, ответственный, группа, контрагент, программа, продукт, договор, файлы с ключами хранилища (backend, bucket, key, sha256) и заявки сайта с их номерами; record.version позволяет получателю отбросить устаревший документ. Контракт приёма — POST /api/crm/interactions с {items} и ответ {accepted, rejected}; заглушки LMS и сайта его реализуют и показывают полученное на GET /api/crm/interactions. LMS даёт обучающихся и потоки, сайт — заявки и их количество за месяц; метрика обновляется по ключу «вуз, программа, месяц, метрика, источник», так что повторная синхронизация не задваивает данные, а источники не затирают друг друга. Заявка с сайта привязывается к взаимодействию по вузу, программе и продукту по умолчанию; если связки ещё нет, взаимодействие заводится на первом этапе у КАМа, который последним работал с этим вузом, а если такого нет — у руководителя команды, которая больше всех ведёт записи этой группы; если нет и его, заявка ждёт человека; если вуз или программа не нашлись в справочниках, заявка остаётся в очереди несопоставленных и ждёт человека. Каждый запуск пишется в sync_run: недоступный источник получает статус failed с кодом INTEGRATION_UNAVAILABLE, а остальные источники синхронизируются как обычно.

События идут в поток Redis с ограничением длины (app/core/events.py): interaction.created, interaction.transitioned, signal.opened, signal.resolved, import.applied, report.updated, notification.created, interaction.status_changed, message.created. Уведомление и сообщение — личные события: его получает только адресат, даже администратор его не видит. Клиент читает их по GET /api/v1/events и получает только события своей области видимости — у каждого события есть владелец записи. Заголовок Last-Event-ID продолжает чтение с места обрыва, раз в 15 секунд приходит комментарий-пульс. Без Redis (тесты, запуск одной командой) та же логика работает в памяти процесса, поэтому поведение потока проверяется тестами без внешних служб. Ночные задачи выполняет воркер arq (app/worker.py): в 03:00 МСК полный пересчёт сигналов, в 03:10 — эскалация зависших записей, следом — подсказки норм по завершённым этапам (медиана и 80-й перцентиль, не меньше пяти наблюдений). Раз в минуту воркер отправляет созревшие уведомления в каналы.

Уведомления (app/modules/notifications/) всегда появляются в ленте сотрудника, а в Telegram, Max или почту уходят, если канал включён администратором, пересылает этот вид уведомлений и сотрудник указал свой адрес. Во внешний канал идут только заголовок события и ссылка: подробности — за входом в систему. Очередь доставки работает как у обмена — захват арендой, отправка вне транзакции и фиксация каждой доставки отдельно, поэтому один неотвечающий канал не откатывает остальные. Владелец записи узнаёт о переходе, который сделал не он (руководитель, групповой переход), и о переносе своих записей при изменении процесса. Запись без изменений дольше срока из настройки stalled_escalation (по умолчанию 14 дней) уведомляет руководителя команды КАМа, а если его нет или запись ведёт он сам — администраторов; один эпизод простоя даёт одно уведомление.

Групповые методы отвечают 200 со списком итогов по каждой записи — успех с новой версией или код ошибки из каталога, поэтому частичный успех считается обычным ответом. expected_version они не принимают: одновременные изменения отсекает блокировка строки, групповой переход разрешён только с этапов с bulk_allowed, а переход с требованием документа выполняется из карточки.

Формат ошибки

```
{
  "type": "about:blank",
  "title": "Нужен комментарий",
  "status": 422,
  "code": "WF_COMMENT_REQUIRED",
  "detail": "Добавьте комментарий: без него переход на этап «Подписание» недоступен.",
  "trace_id": "8f2c1b4e9a0d4c7e",
  "errors": [{ "field": "comment", "message": "Обязательное поле" }]
}
```

Ответы с ошибками объявлены в контракте с типом application/problem+json, и у каждого метода перечислены все коды, которые он может вернуть.

Каталог кодов ошибок

Код	HTTP	Когда
VALIDATION_ERROR	422	Неверные параметры запроса; подробности в errors
AUTH_REQUIRED	401	Нет токена или он просрочен
AUTH_FORBIDDEN	403	Роль не позволяет действие
NOT_FOUND	404	Записи нет или она вне области видимости
METHOD_NOT_ALLOWED	405	Метод не поддерживается по этому адресу
INTERACTION_VERSION_CONFLICT	409	Запись уже изменил другой пользователь
INTERACTION_DUPLICATE	409	С этим контрагентом по той же программе и продукту запись уже ведётся. Сознательное исключение из ADR-007: без него дубль просто не дал бы завести запись, и причина осталась бы непонятной. Ни владельца, ни идентификатор чужой записи ответ не раскрывает
WF_TRANSITION_NOT_ALLOWED	409	Переход в выбранный этап не разрешён правилами
WF_COMMENT_REQUIRED	422	Для перехода нужен комментарий
WF_ATTACHMENT_REQUIRED	422	Для перехода нужен документ
WF_VERSION_NOT_DRAFT	409	В опубликованной версии разрешено только переименование
WF_MIGRATION_MAP_INCOMPLETE	422	Карта переноса ведёт на этап, которого нет в новой схеме
FILE_TYPE_NOT_ALLOWED	415	Тип файла вне белого списка ТЗ
FILE_TOO_LARGE	413	Файл больше лимита
IMPORT_MAPPING_INVALID	422	Маппинг колонок не покрывает обязательные поля
REPORT_TOO_LARGE	422	Отчёт превышает лимит строк
INTEGRATION_UNAVAILABLE	—	По HTTP не возвращается: этим кодом помечается неудачный запуск синхронизации или отправки в sync_run.error_code — LMS или сайт не ответили
INTERNAL_ERROR	500	Непредвиденная ошибка; trace_id для поиска в журнале

Прочие клиентские статусы уровня HTTP, которые поднимает сам фреймворк (например, 400 при испорченном теле запроса), приходят с кодом VALIDATION_ERROR и исходным статусом.

6. Радар и рейтинг

Сигналы

Вид	Условие	Серьёзность
stage_overdue	дней на этапе больше нормы	medium — больше нормы, high — больше двух норм
license_expiring	до license_valid_until 60 дней или меньше	medium — до 60, high — до 30 или истекла
missing_document	этап требует документ, файла нужного типа нет и прошла половина нормы этапа	medium
inactivity	нет переходов, заметок и файлов	low — от 14 дней, medium — от 28

- У сигнала есть evidence: этап, дни, норма и её источник, дата лицензии или последнего действия — всё, чтобы эксперт пересчитал вручную.
- Пороги лицензии и простоя задаёт администратор в настройке radar_thresholds; пока её не сохраняли, действуют значения из таблицы. Новые пороги применяются сразу ко всем активным записям.
- Пересчёт — сразу по событию (переход, файл, договор, импорт) и раз в сутки для всех. Исправленное условие закрывает сигнал (resolved_at).
- Документ обязателен для выхода с этапа: правило перехода требует вложение нужного типа. После перехода сигнал закрывается, а документ остаётся в карточке этапа.
- Нормы задаёт руководитель; когда накопится не меньше 5 завершённых этапов, система предлагает медиану.

Рейтинг востребованности

```
n_m   = 100 × value_m / max(value_m внутри направления)
score = Σ(w_m × n_m) / Σ(w_m)      веса по умолчанию: заявки 40, обучающиеся 40, потоки 20
contrib_m = w_m × n_m / Σ(w_m)     сумма вкладов равна баллу
```

Нет метрики за период — она исключается, веса перераспределяются, позиция помечается «неполные данные».

7. Безопасность (152-ФЗ, приказ ФСТЭК №117)

- Вход: Keycloak, пароль от 12 символов, блокировка после 5 неудачных попыток, простой сессии 30 минут; SPA — Authorization Code + PKCE.
- Доступ: роль + область видимости + правила администратора; тест «чужая запись недоступна» на каждый эндпоинт.

• Журнал: audit_log хранит изменения и просмотр персональных данных, значения до и после.
• Данные: TLS; email и телефон контактов шифруются в приложении; белый список типов файлов; секреты только в окружении; персональные данные не пишутся в логи.
• Целостность: история и аудит только дописываются; ежедневный pg_dump с проверкой восстановления.
• Уязвимости: pip-audit по зависимостям и gitleaks по истории коммитов в CI.
• Данные хранятся на серверах в РФ; на демо-стенде только синтетические данные.

Включение и отключение модулей

Монолит с включаемыми модулями — то, что жюри назвало допустимым. Отдельного механизма фича-флагов в коде нет намеренно: каждый «флаг» здесь означает конкретную настройку, у которой есть владелец и понятное поведение при выключении.

Что выключается	Чем	Что происходит
Канал уведомлений (Telegram, Max, почта)	notification_channel.is_enabled в админке	Уведомления остаются в ленте интерфейса, во внешний канал не уходят
Свой адрес в канале	notification_address.is_enabled	Сотрудник не получает внешних уведомлений, лента остаётся
Исходящий обмен с источником	integration_source.push_enabled в админке	Изменения перестают ставиться в очередь этому получателю; входящая синхронизация продолжается
Входящая синхронизация	integration_source.pull_enabled в админке	Метрики и заявки перестают приходить; рейтинг честно показывает неполные данные
Хранилище файлов	STORAGE_BACKEND=local|s3	Тот же код через интерфейс Storage: каталог на диске для разработки, S3 на стенде
Кэш и поток событий	пустой REDIS_URL	Кэш и события уходят в память процесса: одна копия приложения работает, масштабирование — нет
Хранение персональных данных	пустой PD_ENCRYPTION_KEY	Контакты и почта участников не сохраняются: явный отказ вместо тихого хранения открытым текстом
Вход без Keycloak	AUTH_MODE=dev	Заголовок X-Dev-User вместо токена; в APP_ENV=production запрещено (ADR-011)
Группа контрагентов со своим процессом	группа в админке	Новая группа — новый процесс и свои записи; чужие группы закрываются правилом доступа deny по scope_kind: group

Все переключатели, кроме переменных окружения, доступны администратору из интерфейса и попадают в аудит. Выключенный модуль не ломает остальные: очередь уведомлений продолжает работать без Telegram, а обмен — без сайта.

8. Нефункциональные требования и проверка

Требование	Как проверяем
Отклик ≤ 1 с	k6: p95 < 1000 мс при 50 виртуальных пользователях
10 параллельных отчётов	k6 запускает 10 заданий, все доходят до done
Без перезагрузки	tests/test_events.py: событие перехода приходит в поток SSE тем, кому запись видна
Коды ошибок	Тесты на каждый код каталога
Swagger и перечень библиотек	/api/docs; перечень — backend/requirements.lock и frontend/package.json; CI выкладывает полный список артефактом к каждой сборке
Docker и Linux	docker compose up в CI на Ubuntu

Методы обработки данных

Как система получает числа, которые показывает. Всё считается детерминированно и проверяемо вручную: ни одна оценка не приходит из модели, которую нельзя пересчитать на бумаге.

Радар: четыре сигнала

Сигнал открывается, пока выполняется условие, и закрывается сам, когда условие перестаёт выполняться. У каждого сигнала есть evidence — числа, по которым он построен.

Сигнал	Условие	Серьёзность
stage_overdue	дней на этапе больше нормы	medium — больше нормы, high — больше двух норм
license_expiring	до конца лицензии 60 дней или меньше	medium — до 60 дней, high — до 30 или уже истекла
missing_document	этап требует документ, документа нужного типа нет и прошла половина нормы	medium
inactivity	нет переходов, заметок и файлов	low — от 14 дней, medium — от 28

Пороги лицензии и простоя — настройка администратора radar_thresholds; в таблице значения по умолчанию. Две недели простоя — ориентир, который назвало жюри. После сохранения настройки сигналы всех активных записей пересчитываются сразу.

Условия и ограничения. Дни считаются в UTC по календарю, а не в рабочих днях. Этап без нормы (последний, «Контроль этапов») не даёт просрочки. Документ засчитывается, если он приложен именно на текущем этапе: тот же файл на прошлом этапе сигнал не закроет. Пересчёт идёт сразу после действия и полностью раз в сутки — за ночь запись может получить сигнал без всякого участия человека, просто потому что прошло время.

Состояние записи: пауза, завершение и отмена

Этап процесса и состояние записи — разное. Этап говорит, где сделка по регламенту; состояние — ведётся ли она вообще.

Состояние	Что значит	Куда можно дальше
active	в работе	пауза, завершение, отмена
paused	вуз взял паузу: ждём бюджет, приёмную кампанию, решение ректората	в работу, завершение, отмена
completed	договор отработан	в работу (переоткрытие), отмена
cancelled	запись заведена ошибочно или сделка не состоялась	в работу

Причина обязательна для паузы и отмены: она остаётся в аудите и в уведомлении ответственному.

Что меняется у неактивной записи. Сигналы радара закрываются сами: просрочка этапа и простой считаются только по активным, поэтому приостановленная запись не мешает руководителю видеть настоящие проблемы и не уходит в эскалацию. Переходы по этапам запрещены, пока запись не вернули в работу. Отменённая уходит из рабочих списков, но остаётся в карточке по прямой ссылке, в истории и в отчёте с фильтром по состоянию — история по ТЗ только дописывается, ничего не удаляется.

Возврат в работу. Отменённая запись не занимает связку «контрагент — программа — продукт», поэтому на её месте могла появиться другая. При возврате связка проверяется заново, и занятая даёт 409 INTERACTION_DUPLICATE — иначе в базе оказались бы две записи об одной и той же сделке.

Эскалация зависших записей

Запись считается зависшей, если она активна и её последняя активность (переход, заметка, документ, смена ответственного) старше срока из настройки stalled_escalation — по умолчанию 14 дней, ориентир жюри «одна-две недели». Проверка идёт ночью и по кнопке администратора.

Кому. Настройка задаёт роль, а не человека. Роль manager — руководители команды владельца записи (сотрудники с ролью руководителя в этой команде и назначенный руководитель команды); если таких нет или запись ведёт сам руководитель, уведомление уходит администраторам. Роль admin — сразу администраторам. Владелец записи эскалацию на себя не получает: у него есть сигнал радара.

Что видит получатель. Список получателей проходит через ту же область видимости, что и списки: руководитель, которому запись закрыта правилом доступа, о ней не узнает, а уведомление уходит администраторам, которые её видят. Во внешние каналы отправляется только заголовок и ссылка.

Сколько раз. Один эпизод простоя — одно уведомление каждому получателю: ключ составлен из записи и даты её последней активности. Повторный запуск ничего не дублирует; стоит КАМу поработать с записью и снова пропасть — начнётся новый эпизод.

Ограничения. Срок считается в календарных днях по UTC. Сигнал радара «нет активности» и эскалация настраиваются отдельно: по умолчанию оба срабатывают через 14 дней, но их можно развести, например сначала сигнал КАМу, потом уведомление руководителю.

Изменение процесса: куда переходят записи

Процесс у группы один, поэтому при публикации изменений все открытые записи переходят на новую схему, а не остаются на старой. Этап назначения выбирается так:

1. Если migration_map указывает этап для кода прежней схемы — туда.
2. Если этап с тем же кодом остался — на него. Норма привязана к коду и сохраняется.
3. Если этап удалён — на ближайший предыдущий этап прежней схемы, который остался в новой. Шаг назад выбран сознательно: вернуться безопаснее, чем проскочить этап, требующий документа.
4. Если предыдущих не осталось — на ближайший следующий; если не осталось ни одного — на первый этап новой схемы.

Групповой перевод записей даёт владельцу одну сводку, а не письмо на каждую запись.

Каждый перенос пишется в историю переходом migration с объяснением, например «Этап «Встреча» удалён из процесса: запись перенесена на «Коммуникация»». Если запись оказалась на другом этапе, дни на этапе считаются заново: иначе радар сразу записал бы КАМу просрочку этапа, на который он запись не ставил. Последняя активность не меняется — перенос схемы не считается работой.

Ограничение. Закрытые записи (completed, cancelled) остаются на прежней схеме: это история.

Группы контрагентов и заведение записи

Запись принадлежит группе, а группа — процессу: «Вузы (B2B)» идут по базовому пути из ТЗ, «Частные лица (B2C)» — по своему. Нормы, требования к документам и радар берутся из шаблона процесса записи, поэтому просрочка заявки частного лица считается от нормы «Заявки» (3 дня), а не от вузовского «Поиска контактов».

При создании записи вручную проверяется:

1. Контрагент ровно один — вуз или клиент. Человек как клиент доступен только тем, кто с ним работает; организация — всем.
2. Продукт. Если у программы есть продукты, выбрать нужно один из них. Если продуктов нет, программа продуктонезависимая и запись заводится без продукта.
3. Дубль. Активная запись с тем же контрагентом, программой и продуктом уже есть — ответ INTERACTION_DUPLICATE. Пустой продукт тоже считается значением, поэтому две записи «вуз × программа без продукта» не появятся. Проверка продублирована уникальным индексом: одновременное создание двух одинаковых записей не пройдёт.
4. Ответственный. По умолчанию — автор. Назначить другого может руководитель в пределах команды или администратор.

Запись встаёт на первый этап действующей схемы процесса группы; комментарий автора становится первой строкой истории.

Нормы этапов и подсказки

Норма задаётся руководителем вручную, у каждого процесса свои нормы. Параллельно система смотрит на историю своего процесса: берёт все завершённые
прохождения этапа, считает длительность каждого как промежуток между переходом на этап и следующим переходом, и если наблюдений не меньше пяти, предлагает две величины:

- медиану — типичный срок;
- 80-й перцентиль методом ближайшего ранга — срок, в который укладывается большинство. Именно он становится нормой при принятии подсказки.

Почему перцентиль, а не среднее. Одна запись, пролежавшая полгода, сдвинула бы среднее и сделала норму бессмысленно мягкой. Перцентиль такой выброс отбрасывает: при наблюдениях 4, 6, 8, 10 и 30 дней подсказка равна 10, а не 11,6.

Ограничение. Подсказка не применяется автоматически — норма меняется только решением человека, и смена пишется в аудит.

Рейтинг востребованности

Считается по трём метрикам ТЗ: заявки, обучающиеся, потоки.

```
n_m   = 100 × value_m / max(value_m внутри направления)
score = Σ(w_m × n_m) / Σ(w_m)       веса по умолчанию: заявки 40, обучающиеся 40, потоки 20
contrib_m = w_m × n_m / Σ(w_m)      сумма вкладов равна баллу
```

Нормирование идёт внутри направления. Сравнивать число заявок на DevOps и на блокчейн бессмысленно: аудитории разного размера. Внутри направления лучший получает 100 баллов, остальные — свою долю от него.

Неполные данные. Если метрики за период нет, она исключается, а её вес распределяется между оставшимися. Программа без данных о потоках не проваливается вниз, но помечается как неполная — это видно в ответе рядом с баллом.

Изменение места считается относительно предыдущего периода такой же длины в целых месяцах: метрики помесячные, поэтому период «15 июня — 10 июля» — это июнь и июль, а предыдущий — апрель и май. Месяц на стыке не попадает в оба периода сразу. Если записи тогда не было, изменение не показывается вовсе, а не выдаётся за рост.

Ограничения. Метрики приходят из LMS (обучающиеся, потоки) и с сайта (заявки); источник входит в ключ, поэтому системы не затирают друг друга. Пока источник не синхронизирован, рейтинг считается по тому, что есть, и честно показывает неполноту.

Ручной приоритет. Рейтинг отвечает на вопрос «что востребовано по данным», а решение «что продвигаем в этом году» остаётся за руководителем: у курса есть приоритет от 0 до 100, который правят руководитель и администратор (изменение пишется в аудит). Приоритет поднимает курс в справочнике и в рейтинге с order=priority, но не меняет ни балл, ни место — место в обоих порядках одно и то же, посчитанное по данным. Так ручное решение видно как ручное, а не выдаётся за расчёт.

Этап на конец периода

Отчёт за прошлый период показывает то, что было тогда, а не сегодняшнее состояние. Этап восстанавливается из истории переходов: берётся последний переход с датой не позже конца периода. История только дописывается (триггер БД запрещает изменение и удаление), поэтому прошлое не переписывается задним числом.

Ограничение. Взаимодействие попадает в период, если создано не позже его конца и либо всё ещё активно, либо в периоде был хотя бы один переход.

Импорт выгрузки заказчика

1. Файл разбирается сразу при загрузке, строки сохраняются как есть. Принимаются xlsx, xls и csv.
2. Колонки сопоставляются полям модели: подсказка строится по нормализованным заголовкам — регистр, пробелы и знаки препинания не важны.
3. Предпросмотр даёт каждой строке решение: new, update, conflict (менеджер не найден), needs_program (ПО нет в справочнике) или skip (нет обязательных колонок).
4. Применение создаёт вузы, договоры, взаимодействия и историю с пометкой import.

Кодировка CSV. Сначала BOM (UTF-8 или UTF-16), затем проверка, что файл целиком читается как UTF-8. Если нет — файл пробуется в cp1251, KOI8-R и cp866, и выигрывает кодировка с наибольшим перевесом строчных русских букв над заглавными: в верной кодировке русский текст в основном строчный, а в чужой те же байты превращаются в заглавные буквы и значки. Разделитель определяется по первым двадцати строкам среди ;, ,, табуляции и |; строка Excel sep=; в начале файла учитывается и в данные не попадает. Кавычки и переносы строк внутри ячеек поддерживаются.

Ограничения. Очень короткий файл из одних заглавных букв определение может перепутать — поэтому определённая кодировка показывается в предпросмотре, а кодировку можно указать при загрузке. Файл, который не читается в указанной кодировке целиком, отклоняется, а не импортируется «кракозябрами». В старых xls (Excel 95) без кодовой страницы указанная кодировка тоже применяется; xls 97 и новее хранят текст в Юникоде.

Ключ строки — вуз, продукт и номер договора, поэтому повторная загрузка того же файла обновляет записи, а не плодит дубликаты. Этап при обновлении не откатывается: файл описывает договор, а процесс ведёт КАМ в карточке.

Справочники файлом

Жюри описало работу со справочниками так: данные собираются заранее и подгружаются в закрытом контуре через окно администратора — JSON или таблицей. Загрузка (POST /api/v1/admin/catalogs/{kind}/import) принимает JSON, CSV, XLSX и XLS для вузов, направлений, программ, вендоров, продуктов, связей программ с продуктами и контактов вендоров.

- Колонки узнаются по имени поля (name, direction_code) или по русскому заголовку из выгрузки («Название», «Код направления»), регистр и знаки не важны.
- Запись находится по естественному ключу: вуз — по названию, направление — по коду, программа — по направлению и названию, продукт — по вендору и названию, связь — по программе и продукту. Повторная загрузка того же файла даёт «без изменений», а не дубли; одинаковый ключ дважды в одном файле — ошибка второй строки.
- Пустая ячейка не стирает поле: файл дополняет справочник, а не перезаписывает его. Архивная запись, встретившаяся в файле, возвращается из архива.
- Предпросмотр (dry_run=true, по умолчанию) выполняет всё в точке сохранения и откатывает: итог по каждой строке — создано, изменено (и какие поля), без изменений или ошибка с причиной — точно совпадает с тем, что даст применение. Ошибка одной строки не мешает остальным.
- Удобства для таблиц заказчика: незнакомый вендор в файле продуктов заводится сразу; «да/нет», «true/false», «1/0» понимаются одинаково; апостроф, который наша выгрузка ставит перед формулой, при загрузке снимается.

Выгрузка (GET /api/v1/admin/catalogs/{kind}/export) отдаёт те же колонки в JSON, CSV (UTF-8 с BOM или windows-1251) или XLSX: выгрузил, поправил в Excel, загрузил обратно. Это проверено тестом для всех трёх форматов.

Выгрузка отчётов без потери формата

- xlsx и xls хранят текст в Юникоде; проверено тестом «выгрузили — прочитали нашим же импортом — совпало с JSON».
- CSV — разделитель ; (так его ждёт Excel в русской локали), кодировка на выбор: UTF-8 с BOM (без потерь, Excel открывает сразу) или windows-1251 для старых программ. Символ, которого нет в cp1251, заменяется на ?, а не роняет отчёт; ответ несёт charset в заголовке Content-Type.
- Защита от формул. В xlsx, xls и csv значение, начинающееся с =, +, -, @, табуляции или перевода строки, получает апостроф: название «=HYPERLINK(...)» не станет формулой у того, кто откроет выгрузку. JSON — данные для программ, поэтому значения в нём отдаются как есть.

Заявки с сайта

Заявка привязывается к взаимодействию по вузу, программе и основному продукту программы. Основная пара «программа ↔ продукт» одна у программы и одна у продукта (её же импорт использует, чтобы по продукту из выгрузки найти программу); если продукт у программы единственный, он основной и без флага. Если продуктов несколько, а основной не выбран, заявка ждёт человека.

Если связки ещё нет, взаимодействие заводится на первом этапе и отдаётся КАМу, который последним работал с этим вузом. Если такого нет — руководителю команды, которая ведёт больше всего записей этой группы; если нет и его, заявка ждёт человека. Если вуз или программа не нашлись в справочниках, заявка остаётся в очереди несопоставленных — система не угадывает.

Очередь несопоставленных видят все руководители: её кто-то должен разобрать. Сопоставленная заявка с ФИО заявителя видна только тем, кто видит её взаимодействие, и перепривязать чужую заявку к своей записи нельзя.

Оплаты частных лиц и зачисление в LMS

Кейсодержатель передаёт оплаты выгрузкой платёжной системы — массивом JSON с номером заявки, курсом, ФИО, телефоном, почтой и номером потока (POST /api/v1/payments/import). Каждая запись файла разбирается отдельно:

1. Номер заявки уникален: уже загруженная оплата — «без изменений», повторная загрузка

файла ничего не дублирует. null и записи без обязательных полей — ошибки своих строк.
2. Курс ищется по названию в справочнике программ без учёта регистра и знаков. Продукт курса — основная пара «программа ↔ продукт» или единственный продукт; у продуктонезависимого курса продукта нет.
3. Человек находится по HMAC-отпечатку почты. Карточки, заведённые до появления отпечатка, сверяются по ФИО и расшифрованной почте. Не нашёлся — заводится клиент-человек; почта и телефон шифруются, телефон приводится к виду +7 (999) 023-43-65.
4. Запись — связка «клиент × курс × продукт» в группе «Частные лица». Нет — заводится сразу на этапе «Зачисление в LMS». Есть и стоит раньше — переходит на зачисление; уже дальше — остаётся на своём этапе; на паузе — строка с ошибкой, оплату проводят в карточке. В историю пишется «Оплата по заявке ORD-…, поток N» с источником import.
5. Запись должна попасть в область видимости того, кто загружает, иначе строка — ошибка.

Файл для LMS (GET /api/v1/payments/lms-users) собирает клиентов-людей на этапе зачисления в области видимости, по курсу и номеру потока из оплаты; человек с двумя курсами — одной строкой. Шаблон повторён дословно: 30 колонок первого листа, списки «Пол» и «Образование» на втором листе, выпадающие списки в колонках L и W. Заполняются фамилия, имя и отчество (разбор ФИО по пробелам), телефон числом без «+» (79990234365, как в шаблоне) и почта. Проверено на файлах кейсодержателя: выгрузка по его пяти оплатам совпадает с его «Загрузкой пользователей» по заголовкам, данным, второму листу и спискам.

Контакты вендоров

Таблица кейсодержателя «Вендоры» (компания, продукт, ФИО, телефон, почта, способ связи) грузится импортом справочников vendor-contacts. Вендор находится по названию, продукты — по названию у вендора; в ячейке продукта можно перечислить несколько в кавычках «…», «…». Человек находится по ФИО у вендора, пустая ячейка не стирает известную почту или телефон. Способы связи — «Почта», «Чат в ТГ», «Телефон» в любом сочетании через запятую. Выгрузка возвращает ту же таблицу; телефон с «+» получает апостроф (защита от формул в Excel), при загрузке апостроф снимается.

Списки обучающихся и преподавателей

Список привязан к записи: обучение идёт по связке «вуз — программа — продукт», и люди в нём — те, кого по ней учат, и те, кто ведёт занятия. Отдельного справочника людей нет намеренно: он превратился бы в базу персональных данных, которую никто не чистит.

Как попадают в список. Руками по одному или файлом целиком (JSON, CSV, XLSX, XLS — те же кодировки и тот же предпросмотр, что у справочников). Строка файла находит человека по отпечатку почты, а если почты нет — по ФИО и роли; человека, заведённого раньше без адреса, список с адресами не задваивает, а дописывает ему почту. Источник lms в модели — для составов групп, которые передаёт LMS; заглушка LMS их не передаёт, поэтому на стенде списки ведут люди. Роль берётся из колонки «Роль»
(«обучающийся» или «преподаватель»), а если колонки нет — из выбора при загрузке.

Что с персональными данными. ФИО видит тот, кто видит запись. Почта хранится зашифрованной, в списке показана сокращённо (и***@вуз.рф), целиком отдаётся отдельным запросом, и этот запрос вместе с выгрузкой файла попадает в аудит. Рядом с зашифрованной почтой лежит её HMAC-отпечаток: он нужен, чтобы искать дубли, и по нему адрес не восстановить. Человек ушёл с обучения — строка уходит из списка, а почта стирается.

Чем список не является. Метрика «обучающиеся» в рейтинге приходит из LMS и считает всех по вузу и программе за месяц; список — это конкретные люди конкретной записи. Совпадать они не обязаны: список ведут КАМы, метрику отдаёт LMS. Во внешние системы список не выгружается — в LMS эти люди уже есть.

Кэш: что и насколько живёт

Что	Где лежит	Когда обновляется
Справочники: направления, программы, продукты, группы	Redis, готовый ответ	сразу при изменении справочника в админке или загрузке файлом; иначе через 5 минут
Рейтинг востребованности	Redis, готовый ответ по каждому набору параметров	при смене весов, ручного приоритета и после синхронизации метрик; иначе через 5 минут

Что	Где лежит	Когда обновляется
Карточка взаимодействия, списки справочников, рейтинг	кэш браузера с проверкой на сервере (ETag)	каждый запрос: сервер отвечает 304, если ответ не изменился
Файл вложения, готовый отчёт	кэш браузера с проверкой на сервере (ETag, private, no-cache)	содержимое по адресу не меняется, но доступ проверяется при каждом запросе: потерявший доступ не скачает файл из кэша

Всё, что зависит от прав, помечено private — такой ответ хранит только сам браузер, и то с проверкой на сервере: иначе после сужения области видимости пользователь видел бы старое. ETag карточки считается по телу ответа, поэтому изменение любого её поля — этап, сигнал, заметка, состояние — делает его другим. ETag файла — его SHA-256, отчёта — запуск и время готовности.

Что будет, если Redis недоступен. Ничего: кэш — ускорение, а не источник правды. Любая ошибка означает «в кэше пусто», ответ собирается из базы, а сброс поколения, который не удался, проживёт не дольше пяти минут — столько живёт запись кэша.

Руководство пользователя

Этот файл собирается из встроенной справки командой make help: те же разделы пользователь видит в самом продукте (значок «Справка», методы GET /api/v1/help). Править нужно исходники в [backend/app/help/](../backend/app/help), а не этот файл.

Эксплуатация стенда — в руководстве администратора; как считаются радар, нормы и рейтинг — в методах обработки данных.

Содержание

- С чего начать — Вход, роли и что видно на первом экране
- Как ведётся взаимодействие — Этапы, переходы, комментарии, документы и групповые действия
- Радар: что означают сигналы — Четыре сигнала, пороги и что с ними делать
- Пауза, завершение и отмена — Состояние записи отдельно от этапа процесса
- Обучающиеся и преподаватели — Список группы по записи, загрузка файлом и работа с почтой
- Оплаты частных лиц и зачисление в LMS — Выгрузка платёжной системы, запись B2C и файл «Загрузка пользователей»
- Отчёты и выгрузки — Как заказать отчёт, какие форматы и что с кодировками
- Рейтинг востребованности и приоритеты — Как считается балл и как задать порядок продвижения вручную
- Уведомления и переписка — Лента, внешние каналы и обсуждение записи с коллегой
- Фильтры и сохранённые виды — Как не набирать одни и те же фильтры каждый день
- Что настраивает администратор — Справочники, правила доступа, процессы, каналы и аудит

С чего начать

Кому показывается: admin, kam, manager.

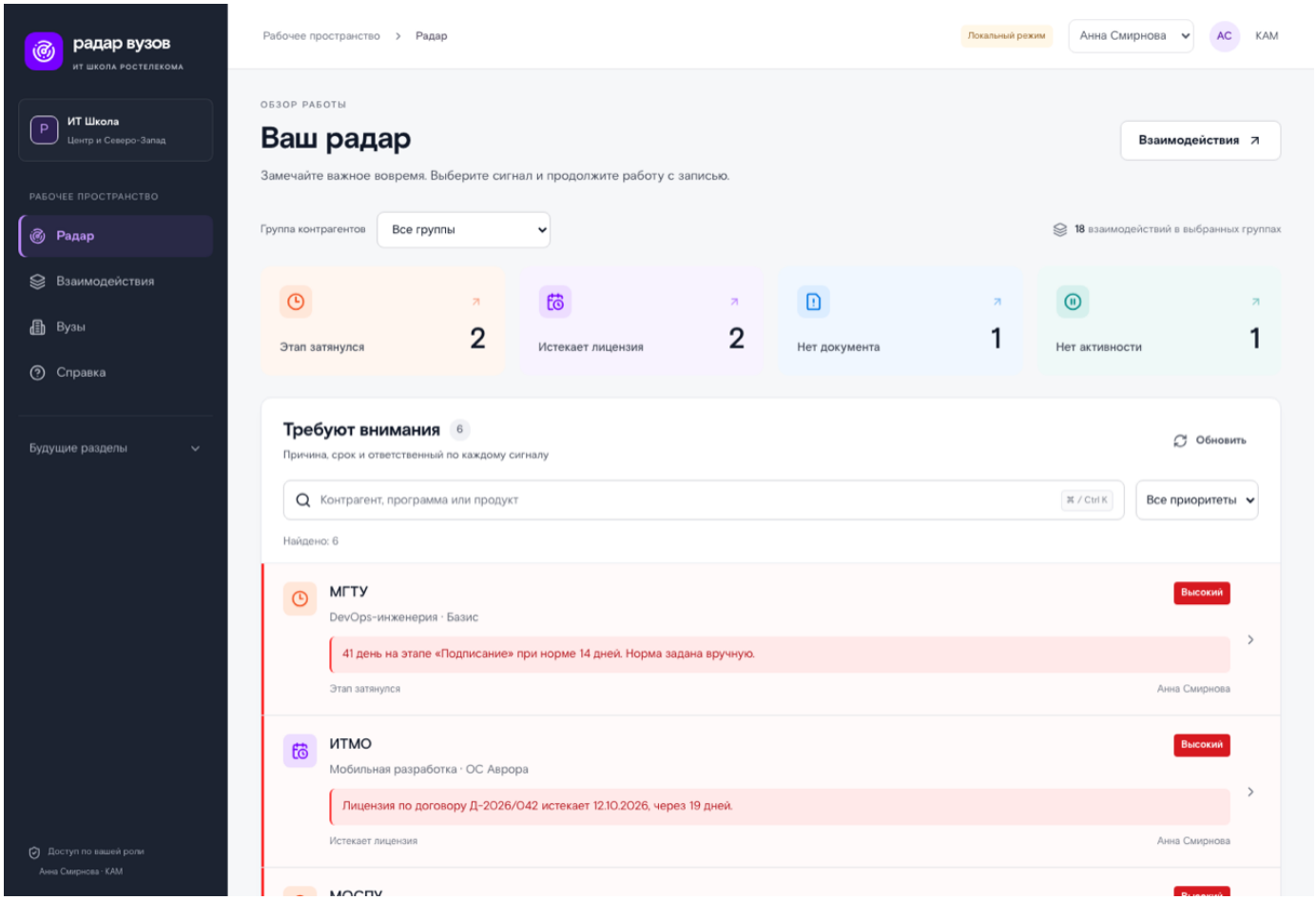
Вход — через корпоративную учётную запись: на странице входа нажмите «Войти», введите логин и пароль, система вернёт вас обратно. Отдельного пароля у «Радара вузов» нет.

Что вы видите, зависит от роли.

- КАМ — свои взаимодействия: вузы и клиенты, с которыми работает он сам.
- Руководитель — всю свою команду: записи её КАМов, их сигналы и отчёты; он же назначает ответственных, задаёт нормы этапов и веса рейтинга.
- Администратор — всё, включая справочники, правила доступа, настройки и журнал аудита.

Область видимости работает везде одинаково: если запись не ваша, она не появится ни в списке, ни в поиске, ни в отчёте, а прямая ссылка на неё покажет «не найдено». Это не ошибка — так закрыты чужие данные.

Первый экран — радар: записи, которые требуют внимания сегодня. Ниже — списки и поиск. Если чего-то не хватает на экране, проверьте фильтры: они запоминаются.



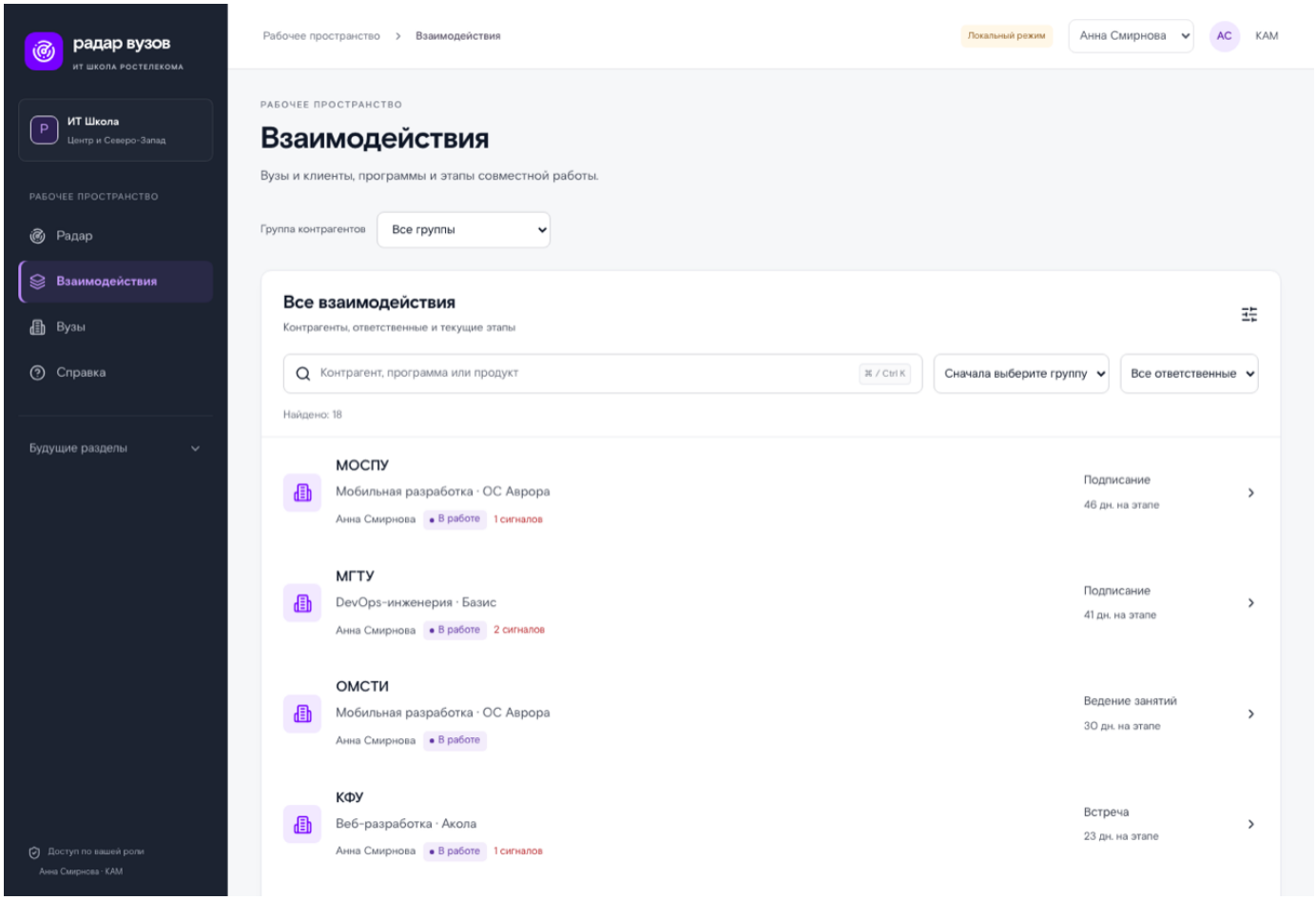
Экран радара: четыре счётчика сигналов и список записей, требующих внимания

Слева — разделы, вверху справа — кто вы и в какой роли работаете.

Как ведётся взаимодействие

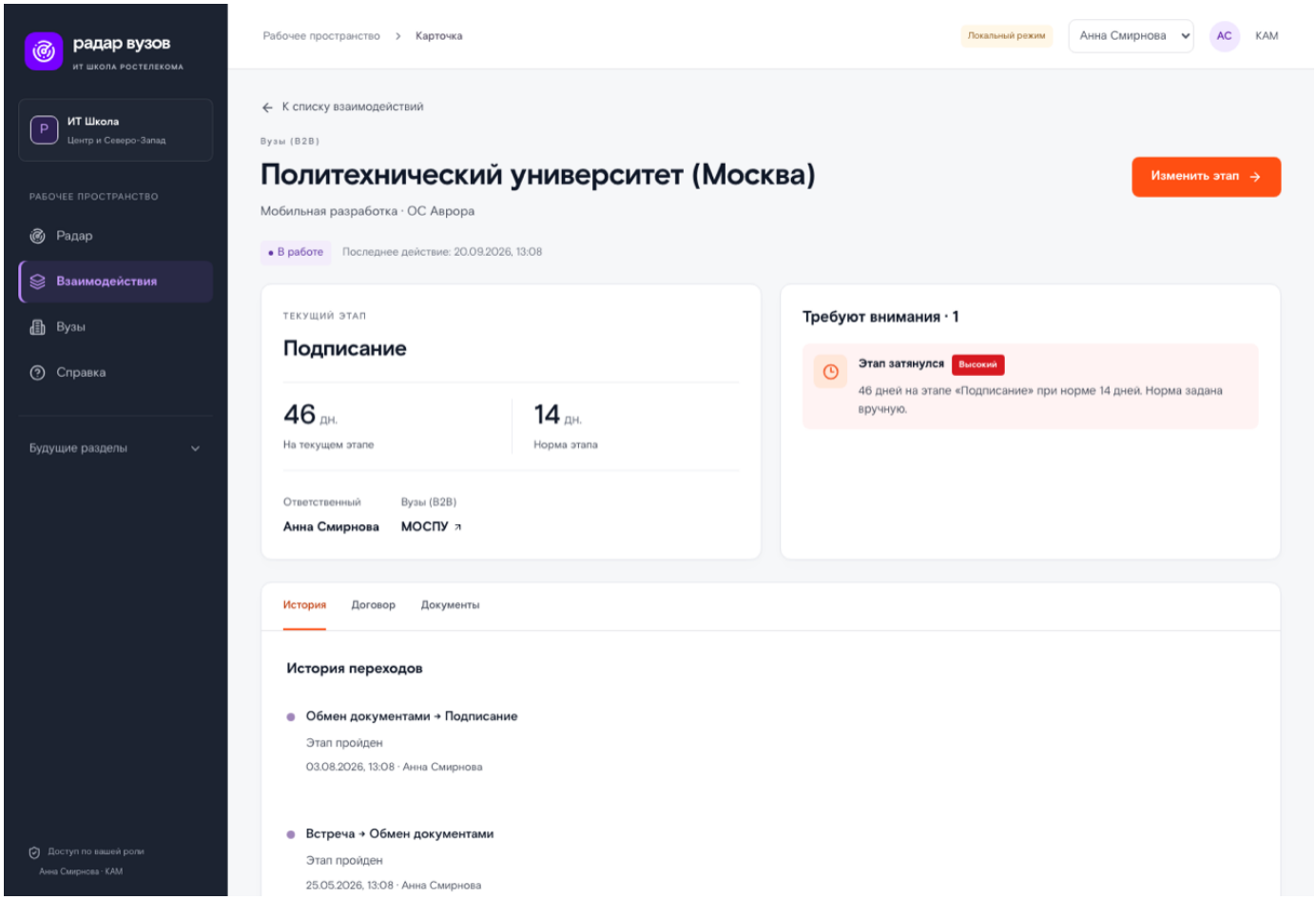
Кому показывается: admin, kam, manager.

Взаимодействие — это связка «контрагент — программа — продукт»: с кем работаем, чему учим и на чём. Контрагент — вуз (B2B) или клиент, человек или организация (B2C). Продукт бывает не нужен: часть программ идёт без него.



Список взаимодействий с фильтрами по группе, этапу и ответственному

Этапы. Запись движется по процессу своей группы: у вузов один путь, у частных лиц — свой. В карточке видно, на каком этапе запись, сколько дней она на нём и куда с него можно перейти. Кнопки показывают только разрешённые переходы — включая возврат на шаг назад, если что-то забыли.



Карточка записи: текущий этап, дни на этапе, норма, сигналы и история переходов

Что требуется для перехода. У перехода может быть обязательный комментарий и обязательный документ. Если документа нет, кнопка перехода откажет и скажет, какого типа документ нужен; приложите его на текущем этапе — файл с прошлого этапа не засчитывается.

Если кто-то изменил запись раньше вас. Система не даст переписать чужое изменение: вы увидите «запись изменена, обновите страницу». Обновите — и повторите действие, уже видя актуальное состояние.

Несколько записей сразу. В списке можно отметить записи и перевести их одним действием, если этап это разрешает, или передать другому КАМу. Система покажет, что получилось, а что нет и почему: одна неудачная запись не отменяет остальные.

Заметки. Короткая заметка в карточке — это и память для себя, и сигнал системе: запись перестаёт считаться простаивающей.

Радар: что означают сигналы

Кому показывается: admin, kam, manager.

Радар показывает записи, где что-то идёт не так. Сигнал открывается сам, пока выполняется условие, и закрывается сам, когда условие перестало выполняться. У каждого сигнала есть числа, по которым он построен, — их видно в карточке.

Сигнал	Когда появляется	Что делать
Просрочка этапа	дней на этапе больше нормы	перевести запись дальше или объяснить задержку заметкой
Лицензия истекает	до конца лицензии 60 дней и меньше	начать продление, не дожидаясь конца
Нет документа	этап требует документ, прошла половина нормы	приложить документ нужного типа
Простой	нет переходов, заметок и файлов	связаться с вузом и оставить заметку

Пороги лицензии и простоя задаёт администратор; по умолчанию простой считается с двух недель, а серьёзным становится через месяц. Сроки этапов — нормы — задаёт руководитель, и система подсказывает ему типичный срок по завершённым этапам.

Записи, по которым долго ничего не происходит, поднимаются выше. Если запись зависла не по вашей вине — поставьте её на паузу с причиной: она перестанет копить сигналы и не будет мешать видеть настоящие проблемы.

Пауза, завершение и отмена

Кому показывается: admin, kam, manager.

Этап говорит, где сделка по регламенту. Состояние — ведётся ли она вообще.

- В работе — обычное состояние.
- Приостановлена — вуз взял паузу: ждём бюджет, приёмную кампанию, решение ректората. Причина обязательна. Пока запись на паузе, сигналы радара по ней не копятся и переходы по этапам недоступны.
- Завершена — договор отработан. Запись можно переоткрыть, если работа возобновилась.
- Отменена — запись заведена ошибочно или сделка не состоялась. Причина обязательна. Отменённая уходит из рабочих списков, но остаётся в истории и в отчёте с фильтром по состоянию: ничего не удаляется.

Возврат в работу. Отменённая запись освобождает связку «контрагент — программа — продукт», поэтому на её месте могла появиться другая. Если так и случилось, вернуть старую не получится: система скажет, что такая запись уже ведётся.

Смена состояния записи видна ответственному: если состояние поменял руководитель, КАМ получит уведомление с причиной.

Обучающиеся и преподаватели

Кому показывается: admin, kam, manager.

У каждой записи есть список тех, кого по ней учат, и тех, кто ведёт занятия. Список живёт рядом с записью: отдельного справочника людей в системе нет намеренно.

Как добавить. По одному — кнопкой «Добавить», указав ФИО, роль и, если есть, рабочую почту. Целиком — файлом: JSON, CSV, XLSX или XLS с колонками «ФИО», «Email», «Роль» и «Идентификатор в LMS». Сначала система покажет предпросмотр по строкам: что появится, что обновится, где ошибка. Применяется только после подтверждения.

Повторная загрузка того же файла ничего не задваивает: человек находится по почте, а если почты нет — по ФИО и роли.

Почта — персональные данные. В списке она показана сокращённо (и***@вуз.рф). Адрес целиком открывается по кнопке, и каждый такой просмотр, как и выгрузка списка файлом, попадает в журнал аудита. Это требование закона, а не придирка системы.

Человек ушёл с обучения — уберите его из списка: строка исчезнет, а почта будет стёрта. Хранить её дальше незачем.

Число обучающихся в списке и метрика «обучающиеся» в рейтинге — разные вещи. Список ведёте вы, метрику отдаёт LMS по вузу и программе за месяц; совпадать они не обязаны.

Оплаты частных лиц и зачисление в LMS

Кому показывается: admin, kam, manager.

Частное лицо проходит свой процесс: заявка, консультация, договор-оферта, оплата, зачисление в LMS, обучение. Оплату не нужно проводить руками по каждой записи — её приносит выгрузка платёжной системы.

Загрузить оплаты (руководитель и администратор). Файл — JSON платёжной системы с полями «Номер заявки», «Курс», «Фамилия», «Имя», «Отчество», «Телефон», «Email», «Номер потока»; подойдёт и таблица с теми же колонками. Сначала система показывает, что сделает с каждой записью, и только по кнопке применяет. Для каждой оплаты она:

- находит человека по почте, а если его ещё нет — заводит карточку клиента;
- находит его запись по курсу, а если её нет — заводит запись в группе «Частные лица»;
- переводит запись на этап «Зачисление в LMS» и пишет в историю номер заявки и поток.

Запись, которая уже дальше зачисления (обучение, аттестация), остаётся на своём этапе — оплата просто добавляется к ней. Приостановленную запись загрузка не трогает: такую оплату проведите в карточке. Повторная загрузка того же файла ничего не дублирует: номер заявки уникален. Пустые и кривые записи файла показываются отдельными строками с причиной.

Курс ищется по названию в справочнике программ. Если курса нет, строка вернётся с ошибкой — добавьте программу в «Каталогах» и загрузите файл ещё раз.

Файл для LMS. Кнопка «Загрузка пользователей» отдаёт XLSX ровно по шаблону LMS: те же колонки, второй лист со списками «Пол» и «Образование». В файл попадают люди на этапе «Зачисление в LMS» из вашей области видимости; можно выбрать курс и номер потока. Заполнены ФИО, телефон и почта — паспорт, СНИЛС и диплом система не хранит, их человек вносит в LMS сам. В файле персональные данные, поэтому каждая выгрузка пишется в журнал аудита.

После зачисления переведите запись на «Обучение» — как обычный переход.

Контакты вендоров

У каждого продукта есть человек со стороны вендора: к нему идут, когда вузу или слушателю нужна помощь с продуктом. Список — в «Каталогах», раздел вендоров: компания, её продукты и контакты с телефоном, почтой и удобным способом связи (почта, Telegram, звонок).

Администратор загружает таблицу вендоров файлом — колонки «Компания», «Продукт», «ФИО», «Телефон», «Почта», «Способ связи». В ячейке продукта можно перечислить несколько продуктов в кавычках: «RT.DataLake», «RT.Warehouse». Недостающие вендоры и продукты заводятся сами, повторная загрузка ничего не дублирует. Почта и телефон хранятся зашифрованными, просмотр и выгрузка контактов пишутся в журнал аудита.

Отчёты и выгрузки

Кому показывается: admin, kam, manager.

Отчёт строится за период и в фоне: вы заказываете его и продолжаете работать, а когда файл готов — забираете. В списке отчётов видно состояние и прогресс.

Что можно настроить. Период, фильтры (группа, вуз, программа, продукт, ответственный, этап, состояние) и набор колонок. Отчёт показывает состояние на конец периода, а не сегодняшнее: этап восстанавливается из истории переходов.

Форматы. XLSX и XLS — для Excel, CSV — для загрузки в другие системы, PDF — чтобы отправить или распечатать, JSON — для интеграции.

Кодировки. CSV выгружается в UTF-8 с меткой (Excel откроет без плясок) или в windows-1251, если принимающая сторона ждёт именно её. Русские буквы не «поедут» ни в одном из вариантов — это проверяется тестами «выгрузили, загрузили обратно, сравнили».

Отчёт не может увидеть больше, чем видите вы: область видимости применяется и к нему.

Рейтинг востребованности и приоритеты

Кому показывается: admin, kam, manager.

Рейтинг отвечает на вопрос «что востребовано по данным». Считается по трём метрикам: заявки с сайта, обучающиеся и потоки из LMS. У каждой метрики свой вес, сумма весов — 100; веса по умолчанию задаёт руководитель, а в самом рейтинге их можно покрутить и сразу увидеть другой порядок.

Сравнение идёт внутри направления. Число заявок на DevOps и на блокчейн сравнивать бессмысленно: аудитории разного размера. Внутри направления лучший получает 100 баллов, остальные — свою долю от него.

Неполные данные не прячутся. Если метрики за период нет, она исключается, её вес распределяется между остальными, а строка помечается как неполная — это видно рядом с баллом.

Изменение места считается относительно предыдущего периода такой же длины. Если раньше записи не было, изменение не показывается вовсе, а не выдаётся за рост.

Ручной приоритет. Решение «что продвигаем в этом году» остаётся за человеком: руководитель

и администратор ставят курсу приоритет от 0 до 100. Отмеченные курсы идут первыми в справочнике и в рейтинге в режиме «по приоритету», но балл и место при этом не меняются — они всегда посчитаны по данным. Так видно, где расчёт, а где решение.

Уведомления и переписка

Кому показывается: admin, kam, manager.

Лента уведомлений — значок в шапке. Туда приходят зависшие записи, переходы, сделанные не вами по вашей записи, смена ответственного и перенос записей при изменении процесса. Прочитанные уходят из непрочитанных, но остаются в ленте.

Внешние каналы. Те же уведомления можно получать в Telegram, Max или на почту. Адрес указывается в профиле, каналы включает администратор. Во внешний канал уходит только заголовок события и ссылка: персональные данные и содержание записи в мессенджер не передаются.

Переписка с коллегой. Внутри системы есть личные сообщения: обсудить запись с руководителем можно, не выходя в сторонний мессенджер. К сообщению прикладывается ссылка на вашу запись — собеседник увидит её, если запись доступна и ему. Переписку видят только двое: ни другой КАМ, ни администратор чужой диалог не откроют. Наружу переписка не пересылается.

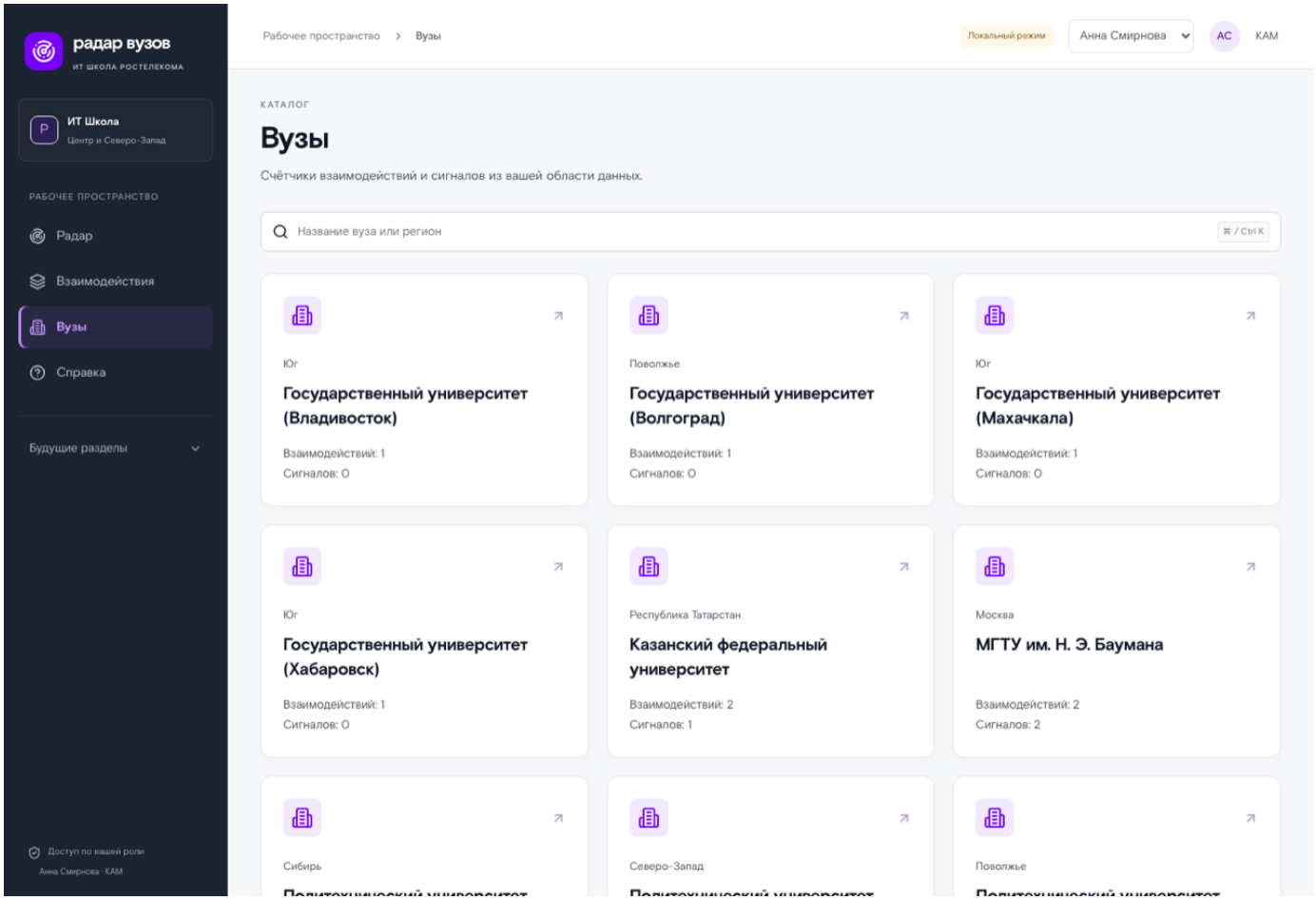
Зависшая запись уходит выше. Если по записи долго ничего не происходит, уведомление получает не конкретный человек, а роль — руководитель команды или администраторы. Срок задаёт администратор.

Фильтры и сохранённые виды

Кому показывается: admin, kam, manager.

Списки фильтруются по группе, вузу или клиенту, направлению, программе, продукту, ответственному, этапу, состоянию, числу дней на этапе и открытым сигналам. Поиск ищет по названию контрагента и программе.

Набор фильтров и колонок можно сохранить под своим названием — например «Мои просрочки» или «B2C в работе». Сохранённые виды личные: у коллеги их нет, даже у администратора. Название в пределах страницы занимается один раз, но на другой странице то же название свободно.



Список вузов со счётчиками взаимодействий и открытых сигналов

Вид хранит настройки списка, а не сами данные: он всегда показывает то, что в системе сейчас.

Что настраивает администратор

Кому показывается: admin.

- Справочники — вузы, направления, программы, вендоры, продукты и их связи. Заводятся по одной записи или файлом: JSON, CSV, XLSX, XLS с предпросмотром по строкам. Повторная загрузка того же файла не создаёт дублей, пустая ячейка не стирает заполненное поле. Выгрузка отдаёт ровно те колонки, которые принимает загрузка.
- Группы контрагентов — «Вузы (B2B)», «Частные лица (B2C)» и новые группы. У каждой свой процесс; новая группа заводится на уже опубликованном процессе.
- Процессы — этапы, их порядок, правила переходов и требования к комментарию и документу. Изменения готовятся черновиком и применяются ко всем открытым записям сразу; перед публикацией окно подтверждения показывает, что переименовано, что удалено и куда переедут записи. Переименование этапа доступно только администратору.
- Правила доступа — точечные разрешения и запреты по вузу, направлению, программе или группе. Запрет сильнее разрешения и убирает записи из всех списков, сигналов и отчётов.
- Настройки — пороги радара и срок, после которого зависшая запись уходит вышестоящей роли.
- Каналы уведомлений — Telegram, Max и почта: включение, адрес и виды событий. Секрет канала хранится в переменных окружения, в базе — только имя переменной.
- Журнал аудита — кто что изменил и кто смотрел персональные данные. Журнал только дописывается.

Эксплуатация стенда — установка, обновление, резервные копии, разбор происшествий — в руководстве администратора репозитория: docs/ADMIN_GUIDE.md.

Руководство администратора

Документ для того, кто ставит систему, следит за ней и отвечает на вопросы пользователей. Развёртывание стенда описано отдельно — [docs/DEPLOY.md](DEPLOY.md), меры защиты — [docs/SECURITY.md](SECURITY.md).

Из чего состоит система

Часть	Что делает	Если остановится
web (nginx)	Отдаёт интерфейс, проксирует API и Keycloak, держит TLS	Система недоступна снаружи
api	REST и поток событий	Интерфейс открывается, но данные не грузятся
worker	Ночной пересчёт радара, подсказки норм, отчёты, синхронизация	Сигналы не обновляются к утру, отчёты копятся в очереди
keycloak	Вход, роли, пароли	Никто не войдёт; уже вошедшие работают, пока живёт токен (5 минут)
postgres	Все данные	Система не работает
S3-хранилище: бакет Object Storage на стенде, minio в профиле без облака	Файлы вложений и отчётов	Документы не загружаются и не скачиваются, отчёты не сохраняются; /api/health отвечает ошибкой
redis	Очередь заданий и поток событий	Отчёты не строятся, доска не обновляется без перезагрузки
backup	Ежедневный дамп	Резервных копий не появляется — заметно только тогда, когда они понадобятся
backup-files (только со своим MinIO)	Ежедневная копия файлов из MinIO; в Object Storage вместо неё — версионирование бакета	Дамп базы есть, а файлов к нему нет: восстановленные записи ссылаются на пустоту
mock-messengers, mailpit	Заглушки Telegram и Max, почтовый сервер для писем	Уведомления копятся в очереди и уходят после восстановления; в ленте интерфейса они видны сразу

Установка

Порядок — в [docs/DEPLOY.md](DEPLOY.md). Коротко: заполнить .env по образцу [infra/env.prod.example](../infra/env.prod.example), поднять профиль стенда, применить миграции, при необходимости загрузить демо-данные и прогнать [infra/smoke.sh](../infra/smoke.sh).

Обязательные секреты: пароль базы, ключ шифрования контактов (PD_ENCRYPTION_KEY), логин и пароль администратора Keycloak. Без любого из них запуск прерывается с указанием, чего не хватает.

Пользователи и доступ

- Учётные записи заводятся в Keycloak. При первом входе сотрудник появляется в системе автоматически: роль берётся из токена, команда назначается администратором.
- Роли: kam — свои взаимодействия; manager — своя команда, назначение ответственных, нормы, импорт, веса рейтинга; admin — всё, включая администрирование и аудит.
- Команда сотрудника меняется в разделе администрирования: PATCH /api/v1/admin/users/{id}.
- Точечные ограничения — правила доступа (/api/v1/admin/access-rules): правило с эффектом deny убирает вуз, направление или программу из всех списков, сигналов и отчётов сотрудника и сильнее любого разрешения. Правило allow наоборот открывает то, что роль не показывает.
- Правило доступа можно адресовать группе контрагентов: например, запретить команде вузов записи частных лиц. Группа — scope_kind: group.
- Группы контрагентов (/api/v1/admin/counterparty-groups). Из коробки — «Вузы (B2B)» и «Частные лица (B2C)», у каждой свой процесс. Новая группа заводится на уже опубликованном процессе: создать шаблон, собрать черновик, опубликовать, затем добавить группу. Процесс группы с открытыми записями не заменяется — его меняют в редакторе, тогда записи переходят на новую схему. Группу с открытыми записями нельзя и архивировать.
- Отключённый сотрудник (is_active: false) не входит и исчезает из фильтров, но его записи и история остаются: владельца нужно сменить вручную.

Ежедневная работа

- Радар пересчитывается сразу после каждого действия (переход, документ, заметка, импорт) и

полностью раз в сутки в 03:00 по Москве. Если сигналы кажутся устаревшими — проверьте, жив ли worker.

• Пороги радара — настройка radar_thresholds (PUT /api/v1/admin/settings/radar_thresholds): за сколько дней предупреждать о лицензии и через сколько дней без изменений считать запись простаивающей. По умолчанию простой — 14 и 28 дней. Сохранение сразу пересчитывает сигналы.

• Изменение процесса. Руководитель готовит черновик, смотрит предпросмотр публикации и публикует; черновик с переименованием этапов публикует администратор, он же переименовывает этапы действующей схемы. Удаление этапа не останавливает работу: записи уходят на соседний этап, а КАМ видит причину в истории карточки.

• Уведомления. Каналы Telegram, Max и почта выключены, пока их не включит администратор (/api/v1/admin/notification-channels). Для канала задаются адрес API или почтового сервера и виды уведомлений; токен или пароль кладётся в переменную окружения с префиксом NOTIFY_ (NOTIFY_TELEGRAM_TOKEN, NOTIFY_MAX_TOKEN, NOTIFY_SMTP_PASSWORD), а в настройке — только её имя.

Адрес API или почтового сервера выбирается из списка NOTIFY_ALLOWED_HOSTS (переменная окружения): канал не должен ходить куда угодно. При смене адреса привязку секрета нужно указать заново.

Кнопка «пробное сообщение» отправляет письмо или сообщение самому администратору сразу, без очереди. Сотрудник указывает свои адреса в профиле. Журнал доставки (/api/v1/admin/notification-deliveries) показывает, что ушло, что ждёт повтора и почему не ушло.

• Эскалация зависших записей — настройка stalled_escalation: срок без изменений (по умолчанию 14 дней) и роль получателя (manager или admin). Проверка идёт ночью; чтобы не ждать, есть POST /api/v1/admin/escalations/run — повторный запуск не дублирует уведомления.

• Нормы этапов задаёт руководитель. Когда по этапу накопится не меньше пяти завершённых прохождений, система предлагает подсказку: медиану и 80-й перцентиль. Подсказка не применяется сама — её принимают кнопкой.

• Отчёты строятся в фоне. Если задание долго висит в состоянии queued, смотрите worker и Redis. Отчёт больше 50 000 строк отклоняется сразу — это защита от заведомо нечитаемой выгрузки.

• Обмен с LMS и сайтом. Входящий идёт раз в час, исходящий — раз в минуту: изменённые записи уходят пакетом JSON. Отправку или приём у конкретного источника администратор выключает (PATCH /api/v1/integrations/{id} с push_enabled: false или pull_enabled: false), например на время работ на стороне получателя. Отправить сразу, не дожидаясь воркера, — POST /api/v1/integrations/{id}/push. Недоступный получатель повторяется без предела попыток с паузой 1, 5, 15, а затем 60 минут: изменения копятся в очереди и уйдут, когда он оживёт. Долгий простой виден по числу попыток в очереди отправки. Токен источника берётся из переменной окружения, имя которой записано в secret_ref.

• Импорт выгрузок заказчика (xlsx, xls или csv): загрузка файла, сопоставление колонок, предпросмотр, применение. Кодировку CSV система определяет сама и показывает в предпросмотре; если вместо букв «кракозябры», файл загружается ещё раз с явной кодировкой.

Повторная загрузка того же файла обновляет записи, а не создаёт дубликаты.

Резервное копирование

• Файлы лежат в S3-хранилище, отдельно от базы: дамп PostgreSQL их не содержит. На стенде в Yandex Cloud это бакет Object Storage с версионированием: удалённый или перезаписанный объект возвращается из прежней версии, отдельная копия не нужна. Ниже — порядок для своего MinIO, когда стенд стоит без облака.

• В своём MinIO (том minio-data) файлы копирует сервис backup-files — раз в сутки mc mirror бакета radar-vuzov в том backups рядом с дампами ([infra/backup/files.sh](../infra/backup/files.sh)). Скрипт сверяет число объектов в хранилище и в копии и падает, если скопировалось меньше. Ключи объектов совпадают с attachment.storage_key и report_job.file_key, поэтому файлы сопоставляются с записями без дополнительных таблиц.

• Копия файлов делается без --remove: файл, удалённый из бакета по ошибке, в ней остаётся. Восстановление файлов — mc mirror /backups/files radar/radar-vuzov в обратную сторону.

• Дамп делается ежедневно для обеих баз — radar и keycloak, хранится 14 дней на стенде и 7 дней локально. Дамп пишется во временный файл и переименовывается в конце: оборванный не выглядит свежей копией.

• Зашифрованные внешние копии и порядок работы с ключом — в DEPLOY.md, раздел «Контроль».

• Проверять восстановление нужно регулярно, а не в день аварии:
docker compose -f docker-compose.prod.yml -f docker-compose.yandex.yml --env-file .env run --rm backup sh /scripts/restore-check.sh
(на сервере без облака — без -f docker-compose.yandex.yml)
разворачивает свежие дампы обеих баз во временные базы, сверяет их с рабочими и удаляет за собой даже при ошибке.
• Восстановление боевой базы: остановить api, worker и keycloak, выполнить pg_restore обоих дампов в чистые базы, поднять сервисы, прогнать infra/smoke.sh.

Обновление

1. Убедиться, что CI на ветке зелёный.
2. Стенд в Yandex Cloud обновляется по шагам раздела «Стенд в Yandex Cloud» → «Обновление» в DEPLOY.md: копия базы, новый код, миграции, перезапуск. Сервер с доступом к репозиторию обновляет рабочий процесс Deploy в GitHub Actions — он делает то же самое и завершается smoke-проверкой.
3. Если smoke упал — откатиться и разобраться по журналам. Порядок важен:
   1. Сначала база, новым образом. Файлы новых миграций есть только в новом образе: старый не знает их ревизий и откатить не сможет. Номер ревизии, на которую возвращаемся, — alembic heads в старом коде (или alembic history в новом).
   docker compose -f docker-compose.prod.yml --env-file .env run --rm migrate alembic downgrade <ревизия>
   2. Потом код: git checkout <прежний коммит> и up -d --build.
   3. Некоторые откаты отказываются работать, если данные нельзя уложить в прежнюю схему (например, записи частных лиц до появления групп). Сообщение говорит, что мешает. Тогда вместо отката — восстановление из дампа, который Deploy снимает перед миграциями.

Откаты, которые затрагивают только данные, данные не удаляют: обратные переходы процесса после отката миграции a41c7e2b9d3f остаются, прежний код с ними работает.

Что смотреть при разборе

• Журнал приложения: код ошибки и trace_id. По trace_id находится вся цепочка запроса.
• Журнал аудита (/api/v1/admin/audit): кто и что менял, со значениями до и после, новые сверху (limit до 500, offset — листать дальше) и с фильтрами по сотруднику (actor_user_id), объекту (entity_kind, entity_id), действию и периоду (occurred_from, occurred_to). Там же фиксируется просмотр контактов — обращение к персональным данным; у смены настройки в записи есть её ключ.
• Журнал обмена (/api/v1/integrations/{id}/runs): почему не пришли метрики или заявки и почему не ушли изменения — у каждого запуска направление pull или push. Очередь отправки (/api/v1/integrations/{id}/outbox) показывает по каждой записи попытки и последнюю ошибку: недоступный получатель повторяется сам, а документ, отклонённый по содержанию, ждёт разбора.
• История переходов в карточке: она только дописывается, поэтому ей можно верить.

Ограничения, о которых стоит знать заранее

• Токен живёт 5 минут, сессия простаивает 30 минут — это политика Keycloak, меняется там же.
• Вложения: не больше 25 МБ, только форматы из ТЗ, тип проверяется по содержимому, а не по имени.
• Персональные данные контактов шифруются ключом PD_ENCRYPTION_KEY. **Потеря ключа означает потерю email и телефонов**: расшифровать их будет нечем, остальные данные не пострадают.
• Каталоги не удаляются, а архивируются: на них ссылаются взаимодействия и история.
• Справочники удобнее вести файлом: выгрузить (/api/v1/admin/catalogs/{kind}/export), поправить в Excel и загрузить обратно (/import) — сначала предпросмотр, затем применение. Порядок для пустой базы: направления → программы → вендоры и продукты → связи программ с продуктами; вузы — в любой момент.

Сборка, установка и стенд

Пошагово для Yandex Cloud (файлы в Object Storage, адрес sslip.io) — раздел «Стенд в Yandex Cloud» ниже.

Профиль стенда — [infra/docker-compose.prod.yml](../infra/docker-compose.prod.yml): nginx с TLS, API, воркер, Keycloak, PostgreSQL, Redis и ежедневный бэкап. Значений по умолчанию у секретов нет: compose не запустится, пока переменная не задана.

Что нужно на сервере

- Docker и плагин compose.
- Каталог /srv/radar/hackathon_2026 с клоном репозитория.
- Сертификаты TLS (например, Let's Encrypt) — в каталоге, который указан в CERTS_DIR; внутри ожидаются fullchain.pem и privkey.pem.
- Сборка фронтенда (npm run build) — в каталоге из SPA_DIR. Keycloak на стенде живёт за nginx по пути /auth, поэтому сборке нужны VITE_AUTH_MODE=keycloak и VITE_KEYCLOAK_URL=https://<домен>/auth; без них интерфейс пойдёт за входом на 127.0.0.1:8080 из примера (frontend/.env.example). VITE_API_BASE_URL остаётся пустым: API на том же адресе, что и интерфейс.

Первый запуск

```
cd /srv/radar/hackathon_2026/infra
cp env.prod.example .env       # заполнить: три пароля базы, ключ шифрования, Keycloak, адреса
docker compose -f docker-compose.prod.yml --env-file .env up -d postgres redis minio
docker compose -f docker-compose.prod.yml --env-file .env run --rm migrate
docker compose -f docker-compose.prod.yml --env-file .env up -d --build
docker compose -f docker-compose.prod.yml --env-file .env run --rm api python -m scripts.seed --full
set -a && . ./.env && set +a   # PUBLIC_URL и прочее — в окружение этой оболочки
sh smoke.sh "$PUBLIC_URL"      # без адреса скрипт проверял бы 127.0.0.1:8000, а не стенд
```

Образы в профиле стенда закреплены по digest (образ:тег@sha256:…): тег можно перезаписать, а стенд должен подниматься тем же, что проверяли. Зависимости Python — по backend/requirements.lock, обновляются командой make lock.

Пароли демо-входов стенда — переменные DEMO_KAM_PASSWORD, DEMO_MANAGER_PASSWORD, DEMO_ADMIN_PASSWORD: без них compose не запустится. Keycloak подставляет их в realm при первом импорте; локальные пароли из realm-radar-vuzov.json на стенде не действуют.

Роли базы

При создании пустого тома PostgreSQL выполняет [infra/postgres/roles.sql](../infra/postgres/roles.sql):

Роль	Кто под ней работает	Что может
radar	сервис migrate, резервные копии	владелец схемы: миграции, DDL
radar_app	api, worker, загрузка демо-данных	только данные; история переходов и аудит — чтение и добавление
keycloak	Keycloak	только своя база keycloak

Приложение не владеет таблицами, поэтому даже с его паролем нельзя отключить триггеры, очистить историю или удалить таблицу. Миграции идут отдельным одноразовым сервисом migrate до старта API и воркера; образ приложения сам их больше не запускает.

Если том создан до появления roles.sql, роли заводятся один раз вручную теми же командами из файла (psql -v app_password=... -v keycloak_password=...), после чего повторно применяется миграция 5c0d9e3a7b21: alembic downgrade 8a1aab17784f && alembic upgrade head под radar.

Обновление

Стенд в Yandex Cloud обновляется по шагам раздела «Стенд в Yandex Cloud» → «Обновление» ниже.
Сервер с доступом к репозиторию обновляется через GitHub Actions: рабочий процесс Deploy, запуск вручную (workflow_dispatch), с флажком
«загрузить демо-данные», если стенд поднимается заново. Он подключается по SSH, обновляет код, собирает образы, снимает дамп обеих баз, прогоняет миграции сервисом migrate, поднимает остальное и завершается проверкой [infra/smoke.sh](../infra/smoke.sh). У рабочего процесса только право чтения репозитория.

Секреты репозитория, которые для этого нужны:

Секрет	Зачем
DEPLOY_HOST, DEPLOY_USER, DEPLOY_KEY	Доступ по SSH к серверу стенда
PUBLIC_URL	Адрес стенда для smoke-проверки и ежечасного контроля

Хранилище файлов

На стенде в Yandex Cloud файлы лежат в Object Storage (раздел «Стенд в Yandex Cloud»). В профиле без облака вложения и отчёты хранятся в MinIO из того же compose; логин и пароль — MINIO_ROOT_USER и MINIO_ROOT_PASSWORD в .env. Бакет radar-vuzov создаётся при первой загрузке файла, наружу MinIO не публикуется. Чтобы перейти на облачное S3-хранилище, достаточно поменять у api и worker переменные S3_ENDPOINT, S3_ACCESS_KEY, S3_SECRET_KEY, S3_BUCKET, S3_SECURE=true и, если нужно, S3_REGION.

Заглушки внешних систем

На стенде LMS, сайт и мессенджеры заменены заглушками из того же образа (mock-lms, mock-site, mock-messengers), а письма принимает Mailpit. Наружу они не публикуются. Посмотреть, что ушло:

- журнал доставки в админке — GET /api/v1/admin/notification-deliveries;
- письма — через туннель ssh -L 8025:127.0.0.1:8025 <сервер> и http://127.0.0.1:8025;
- сообщения мессенджеров — docker compose exec api python -c "import httpx; print(httpx.get('http://mock-messengers:8102/sent').text)".

Для настоящих систем меняются переменные LMS_BASE_URL, SITE_BASE_URL, TELEGRAM_API_URL, MAX_API_URL, SMTP_HOST, SMTP_PORT и секреты NOTIFY_*; код не меняется.

Контроль

- Рабочий процесс Health раз в час прогоняет тот же smoke-скрипт. Пока секрет PUBLIC_URL не задан, он завершается успешно с пояснением: проверять нечего.
- Бэкап делается ежедневно и хранится 14 дней: дампы обеих баз — radar и keycloak. Без второй восстановленный стенд не пустит ни одного пользователя. Проверка восстановления — [infra/backup/restore-check.sh](../infra/backup/restore-check.sh): разворачивает оба дампа во временные базы, сверяет число таблиц с рабочей базой, ищет realm и его пользователей.
- Файлы из MinIO копирует сервис backup-files — тоже ежедневно, в том же томе backups ([infra/backup/files.sh](../infra/backup/files.sh)). Без этой копии дамп базы бесполезен: вложения и готовые отчёты лежат отдельно.
- Внешние копии. Копия на том же сервере не спасает от потери сервера. Если задан BACKUP_GPG_PUBLIC_KEY, backup.sh шифрует дампы и архив файлов открытым ключом в backups/offsite, а files.sh отвозит их в S3-хранилище из OFFSITE_*. Ключ создаётся на своей машине (gpg --quick-generate-key radar-backup), на сервер кладётся только открытый (infra/backup-keys/backup.asc); закрытый хранится отдельно, иначе утечка сервера раскрывает и копии. Расшифровка: gpg --decrypt radar-<дата>.dump.gpg > radar.dump.

Стенд в Yandex Cloud

Пошагово: от промокода до работающего https://<ip>.sslip.io.

Что получится: одна виртуальная машина с Docker (nginx, API, воркер, Keycloak, PostgreSQL, Redis, заглушки LMS, сайта и мессенджеров), файлы вложений и отчётов — в бакете Yandex Object Storage, сертификат Let's Encrypt на бесплатное имя sslip.io, которое указывает на IP машины.

Команды с пометкой на Mac выполняются у себя, на сервере — по SSH.

1. Облако

1. Промокод. Консоль → «Биллинг» → платёжный аккаунт → «Промокоды» → активировать. Грант ложится на платёжный аккаунт, к которому привязано облако.
2. CLI (на Mac, удобнее консоли):

```bash
curl -sSL https://storage.yandexcloud.net/yandexcloud-yc/install.sh | bash
exec -l $SHELL
```

```
yc init            # вход через браузер, выбрать облако, каталог и зону ru-central1-a
```

3. Статический IP — чтобы адрес sslip.io не менялся при перезапуске машины:

```bash
yc vpc address create --name radar-ip --external-ipv4 zone=ru-central1-a
yc vpc address get radar-ip --format json | grep '"address"'
```

4. Машина: Ubuntu 24.04, 2 vCPU (100%), 8 ГБ памяти, 40 ГБ SSD. Меньше памяти не стоит: Keycloak и сборка образа на 4 ГБ упираются в своп.

```bash
ssh-keygen -t ed25519 -f ~/.ssh/radar -N ""          # если ключа ещё нет
yc compute instance create --name radar --zone ru-central1-a \
  --platform standard-v3 --cores 2 --core-fraction 100 --memory 8 \
  --create-boot-disk image-folder-id=standard-images,image-family=ubuntu-2404-lts,size=40,type=network-ssd \
  --network-interface subnet-name=default-ru-central1-a,nat-ip-version=ipv4,nat-address= \
  --ssh-key ~/.ssh/radar.pub
```

Пользователь на машине — yc-user. Если в сети включены группы безопасности, откройте входящие 22, 80 и 443.

5. Бакеты и ключ доступа. Имена бакетов глобальные — добавьте свой суффикс.

```bash
yc storage bucket create --name radar-vuzov-files-<суффикс>
yc storage bucket update --name radar-vuzov-files-<суффикс> --versioning versioning-enabled
yc storage bucket create --name radar-vuzov-backups-<суффикс>
yc iam service-account create --name radar-storage
SA_ID=$(yc iam service-account get radar-storage --format json | python3 -c 'import json,sys; print(json.load(sys.stdin)["id"])')
yc resource-manager folder add-access-binding $(yc config get folder-id) \
  --role storage.editor --subject serviceAccount:$SA_ID
yc iam access-key create --service-account-name radar-storage
```

Последняя команда печатает key_id и secret — секрет показывается один раз, сохраните. Версионирование бакета файлов — это его резервная копия: удалённый или перезаписанный объект можно вернуть. Бакет копий хранит зашифрованные дампы базы (шаг 7).

2. Сервер

```
ssh -i ~/.ssh/radar yc-user@<IP>                          # на Mac
curl -fsSL https://get.docker.com | sudo sh               # на сервере
sudo usermod -aG docker yc-user && sudo mkdir -p /srv/radar && sudo chown yc-user /srv/radar
exit                                                      # перезайти, чтобы группа docker применилась
```

3. Код

Репозиторий приватный, поэтому на сервер едет архив ветки main, а не git clone:

```
cd /Users/olegbragin/Desktop/hackaton/hackathon_2026      # на Mac
git fetch origin && git archive --format=tar.gz -o /tmp/radar.tgz origin/main
scp -i ~/.ssh/radar /tmp/radar.tgz yc-user@<IP>:/srv/radar/
ssh -i ~/.ssh/radar yc-user@<IP> 'mkdir -p /srv/radar/hackathon_2026 && tar -xzf /srv/radar/radar.tgz -C /srv/radar/hackathon_2026'
```

4. Настройки

На сервере, в каталоге infra. Скрипт сам генерирует пароли базы, ключ шифрования и пароли демо-входов и печатает их — сохраните пароли демо-входов для показа.

```
cd /srv/radar/hackathon_2026/infra
YC_S3_ACCESS_KEY=<key_id> YC_S3_SECRET_KEY=<secret> \
YC_S3_BUCKET=radar-vuzov-files-<суффикс> YC_OFFSITE_BUCKET=radar-vuzov-backups-<суффикс> \
```

```
sh yandex/make-env.sh <IP>
```

Адрес стенда будет https://<IP через дефисы>.sslip.io, например https://51-250-10-20.sslip.io. Существующий .env скрипт не перезаписывает: с его паролями уже создана база.

5. Сертификат

```
sh yandex/tls.sh issue          # на сервере, пока nginx не запущен: certbot сам слушает 80 порт
```

Если Let's Encrypt отказал из-за лимита на домен sslip.io, то же самое работает с nip.io: поменяйте в .env PUBLIC_URL на https://<IP через дефисы>.nip.io и повторите.

Продление — раз в неделю через cron (crontab -e):

```
0 4 * * 1 cd /srv/radar/hackathon_2026/infra && sh yandex/tls.sh renew >> /srv/radar/tls.log 2>&1
```

6. Интерфейс

Фронтенд собирается на Mac с адресом Keycloak стенда и отвозится готовым:

```
cd /Users/olegbragin/Desktop/hackaton/hackathon_2026/frontend      # на Mac
npm ci
VITE_AUTH_MODE=keycloak VITE_KEYCLOAK_URL=https://<адрес>.sslip.io/auth npm run build
ssh -i ~/.ssh/radar yc-user@<IP> 'mkdir -p /srv/radar/spa'
scp -i ~/.ssh/radar -r dist/. yc-user@<IP>:/srv/radar/spa/
```

7. Запуск

На сервере, в /srv/radar/hackathon_2026/infra:

```
alias dc='docker compose -f docker-compose.prod.yml -f docker-compose.yandex.yml --env-file .env'
dc build
dc up -d postgres redis
dc run --rm migrate
dc up -d
dc run --rm api python -m scripts.seed --full          # демо-данные, только на пустой базе
set -a && . ./.env && set +a && sh smoke.sh "$PUBLIC_URL"
```

Smoke-скрипт проверяет API, базу, хранилище (бакет), Swagger, Keycloak и то, что без входа API отвечает 401. Интерфейс — https://<адрес>.sslip.io, вход — kam.demo, manager.demo, admin.demo с паролями из шага 4.

Внешние копии базы (по желанию, но без них потеря сервера — потеря данных). На Mac:

```
gpg --quick-generate-key radar-backup
gpg --export --armor radar-backup > /tmp/backup.asc
ssh -i ~/.ssh/radar yc-user@<IP> 'mkdir -p /srv/radar/hackathon_2026/infra/backup-keys'
scp -i ~/.ssh/radar /tmp/backup.asc yc-user@<IP>:/srv/radar/hackathon_2026/infra/backup-keys/
```

На сервере в .env — BACKUP_GPG_PUBLIC_KEY=/keys/backup.asc, затем dc --profile offsite up -d. Закрытый ключ остаётся только у вас: им расшифровывается копия (gpg --decrypt).

8. Обновление

```
git fetch origin && git archive --format=tar.gz -o /tmp/radar.tgz origin/main     # на Mac
scp -i ~/.ssh/radar /tmp/radar.tgz yc-user@<IP>:/srv/radar/
```

```
cd /srv/radar/hackathon_2026 && tar -xzf /srv/radar/radar.tgz && cd infra          # на сервере
dc build && dc run --rm backup sh /scripts/backup.sh && dc run --rm migrate && dc up -d
```

Архив распаковывается поверх: infra/.env, сертификаты и данные лежат вне архива и не затираются. Интерфейс пересобирается и отвозится так же, как в шаге 6.

Что посмотреть, если не работает

- dc ps — все ли контейнеры running, migrate — exited (0).
- dc logs --tail 50 api — storage в /api/health не ok: неверные ключи или имя бакета, или у сервисного аккаунта нет роли storage.editor.
- Вход крутится на странице Keycloak: интерфейс собран не с тем VITE_KEYCLOAK_URL.

- dc logs keycloak — Keycloak стартует до минуты; до этого nginx отвечает 502 на /auth/.

Состояние

Стенд работает в Yandex Cloud, развёрнут по разделу «Стенд в Yandex Cloud»: https://158-160-195-122.sslip.io. Одна машина в зоне ru-central1-d (2 vCPU, 8 ГБ, 20 ГБ), сертификат Let's Encrypt с еженедельным продлением, файлы в бакете Object Storage с версионированием, демо-данные v1, ежедневный дамп обеих баз. infra/smoke.sh по адресу стенда проходит целиком.

16.10 физический сервер облака под первой машиной (ru-central1-a) вышел из строя: машина перестала отвечать, а перезапуск и остановку облако отклоняло. Стенд поднят заново на новой машине по тем же шагам примерно за полчаса: файлы вложений остались в бакете, демо-данные воспроизводятся seed --full, пароли демо-входов перенесены. Поэтому сменился адрес стенда.

Профиль без облака (свой MinIO, этот документ выше) проверен на локальной машине: конфиг nginx проходит nginx -t, бэкап и восстановление прогнаны на локальной базе, копия файлов — на локальном MinIO.

Меры защиты

Матрица мер по 152-ФЗ и приказу ФСТЭК России №117. Каждая строка ссылается на файл, где мера действительно реализована: документ описывает систему, а не намерения.

Вход и учётные записи

Мера	Как сделано	Где
Пароль не короче 12 символов и не совпадает с логином	Политика realm length(12) and notUsername	[infra/keycloak/realm-radar-vuzov.json](../infra/keycloak/realm-radar-vuzov.json)
Блокировка после 5 неудачных попыток	bruteForceProtected, failureFactor: 5, ожидание до 15 минут	там же
Простой сессии 30 минут	ssoSessionIdleTimeout: 1800, время жизни токена 300 с	там же
Самостоятельная регистрация закрыта	registrationAllowed: false	там же
Вход SPA без секрета в браузере	Публичный клиент radar-web, Authorization Code + PKCE S256	там же
Токен проверяется по подписи, издателю и аудитории	decode_keycloak_token	[backend/app/core/security.py](../backend/app/core/security.py)
Режим без Keycloak запрещён в продакшене	Проверка AUTH_MODE=dev при APP_ENV=production	[backend/app/core/config.py](../backend/app/core/config.py)

Разграничение доступа

Мера	Как сделано	Где
Права проверяет только сервер	Область видимости применяется в сервисном слое ко всем запросам	[backend/app/core/scope.py](../backend/app/core/scope.py)
Запись вне области не раскрывается	Ответ 404 NOT_FOUND, а не 403 — существование чужой записи не подтверждается	[backend/app/core/errors.py](../backend/app/core/errors.py)
Правила администратора поверх ролей	data_access_rule: разрешение расширяет область, запрет вычитает и сильнее разрешения	[backend/app/core/scope.py](../backend/app/core/scope.py)
Действия по ролям	require_roles на изменении норм, импорте, интеграциях, весах рейтинга, администрировании	[backend/app/core/security.py](../backend/app/core/security.py)
Уведомления личные	Ленту и событие SSE notification.created видит только адресат, чужое уведомление — 404	[backend/app/modules/events/router.py](../backend/app/modules/events/router.py)
Во внешние каналы не уходят персональные данные	В Telegram, Max и почту отправляются заголовок события и ссылка, без контрагента, ответственного и комментариев	[backend/app/modules/notifications/service.py](../backend/app/modules/notifications/service.py)
Канал ходит только по разрешённым адресам	Хост API мессенджера или почтового сервера сверяется с NOTIFY_ALLOWED_HOSTS; смена адреса отвязывает секрет, а пароль SMTP не уходит без STARTTLS	[backend/app/modules/notifications/channels.py](../backend/app/modules/notifications/channels.py)
Контакты клиента видит тот, кто с ним работает	Карточка организации видна всем, чтобы не плодить дубли, но email и телефон расшифровываются только создателю, его команде и владельцам записей — и каждый такой просмотр пишется в аудит	[backend/app/modules/clients/service.py](../backend/app/modules/clients/service.py)
Отчёт из очереди подчиняется правилам доступа	Воркер строит отчёт с правилами заказчика, а не с пустыми	[backend/app/worker.py](../backend/app/worker.py)
Секреты каналов уведомлений	В базе — только имя переменной, и только с префиксом NOTIFY_: через настройку канала нельзя отправить наружу DATABASE_URL или ключ шифрования	[backend/app/modules/notifications/channels.py](../backend/app/modules/notifications/channels.py)

Персональные данные

Мера	Как сделано	Где
Email и телефон контактов шифруются	Fernet по ключу PD_ENCRYPTION_KEY; без ключа контакт с ПДн не сохраняется	[backend/app/core/crypto.py](../backend/app/core/crypto.py)
Email и телефон клиентов B2C шифруются тем же ключом	ИНН хранится только у организаций: у человека это лишние персональные данные	[backend/app/modules/clients/service.py](../backend/app/modules/clients/service.py)

Мера	Как сделано	Где
Просмотр ПДн фиксируется	Записи contact.viewed, client.viewed, participant.contact_viewed и participant.exported в аудит; в журнал попадает факт, не сами данные	[backend/app/modules/admin/service.py](../backend/app/modules/admin/service.py)
Почта обучающихся и преподавателей шифруется, а в списке показана сокращённо	и***@вуз.рф в списке, адрес целиком — отдельным запросом с записью в аудит; рядом лежит HMAC-отпечаток, по которому загрузка файла находит дубли, но адрес из него не восстановить	[backend/app/modules/participants/service.py](../backend/app/modules/participants/service.py)
Ушедшего из группы убирают вместе с почтой	Строка уходит из списка, email_enc и отпечаток стираются: хранить их больше незачем	[backend/app/modules/participants/service.py](../backend/app/modules/participants/service.py)
Человека видят только те, кто с ним работает	Карточку клиента-человека видят создатель, владельцы его записей, руководитель их команды и администратор; организации видны всем, чтобы не заводить дубли	[backend/app/modules/clients/service.py](../backend/app/modules/clients/service.py)
Переписка сотрудников видна только двоим	Сообщение читают отправитель и получатель, включая администратора — чужой диалог для него пустой; ссылка на запись появляется только у того, кому запись доступна	[backend/app/modules/messages/service.py](../backend/app/modules/messages/service.py)
ПДн не попадают в логи и тексты ошибок	Ошибки отдаются кодом каталога и trace_id; стек — только в журнал сервера	[backend/app/core/errors.py](../backend/app/core/errors.py)
В репозитории только синтетические данные	Демо-пользователи на example.com, фикстуры генерируются	[backend/app/demo.py](../backend/app/demo.py)
Выгрузка не превращается в формулы	В xlsx, xls и csv значения с =, +, -, @ в начале экранируются апострофом (CSV-инъекция)	[backend/app/modules/reports/renderers.py](../backend/app/modules/reports/renderers.py)
Файлы отдельно от базы	Вложения в S3-хранилище без публичного доступа (на стенде — приватный бакет Yandex Object Storage с версионированием); скачивание только через API с проверкой области видимости записи	[backend/app/core/storage.py](../backend/app/core/storage.py)
Потерявший доступ не скачает файл из кэша браузера	Вложения и отчёты отдаются с private, no-cache и ETag: браузер каждый раз спрашивает сервер, а тот сначала проверяет доступ; immutable — только у публичных картинок справки	[backend/app/core/http_cache.py](../backend/app/core/http_cache.py)
ФИО заявителя с сайта видит тот, кто видит запись	Очередь несопоставленных заявок — всем руководителям, сопоставленные — в области видимости их записи; чужую заявку не перепривязать	[backend/app/modules/integrations/service.py](../backend/app/modules/integrations/service.py)
Оплаты без лишних персональных данных	Паспорт, СНИЛС и диплом из шаблона LMS CRM не хранит — колонки пустые; в таблице оплат только номер заявки, поток и ссылки; в журнал уходят номер заявки и поток, не ФИО и не почта	[backend/app/modules/payments/](../backend/app/modules/payments)
Файл зачисления и контакты вендоров — под учётом	Выгрузка «Загрузка пользователей», просмотр и выгрузка контактов вендоров пишутся в аудит; почта и телефоны вендоров зашифрованы	[backend/app/modules/vendors/service.py](../backend/app/modules/vendors/service.py)
Персональные данные участников не хранятся вечно	Через 180 дней после завершения записи ФИО и почта участников стираются ночным заданием, в аудит пишется факт	[backend/app/modules/participants/service.py](../backend/app/modules/participants/service.py)

Журналирование и целостность

Мера	Как сделано	Где
Журнал изменений со значениями до и после	audit_log с полями before и after	[backend/app/modules/audit/models.py](../backend/app/modules/audit/models.py)

Мера	Как сделано	Где
История и аудит только дописываются	Триггеры БД запрещают UPDATE, DELETE и TRUNCATE и работают в режиме ALWAYS — их не отключает и session_replication_role; у роли приложения на эти таблицы нет прав UPDATE, DELETE, TRUNCATE; на всё есть тесты	[backend/migrations/versions](../backend/migrations/versions)
Каталоги не удаляются	Архивирование archived_at вместо удаления	[backend/app/modules/admin/service.py](../backend/app/modules/admin/service.py)
Параллельные изменения не затирают друг друга	expected_version и блокировка строки при переходах	[backend/app/modules/interactions/service.py](../backend/app/modules/interactions/service.py)

Файлы и внешние данные

Мера	Как сделано	Где
Белый список типов файлов	Проверяются и расширение, и сигнатура: exe с именем .pdf отклоняется	[backend/app/modules/attachments/files.py](../backend/app/modules/attachments/files.py)
Предел размера загрузки	MAX_UPLOAD_MB (25 МБ по ТЗ), превышение — FILE_TOO_LARGE	[backend/app/modules/attachments/service.py](../backend/app/modules/attachments/service.py)
Имя файла обезврежено	Путь и управляющие символы вырезаются перед сохранением	[backend/app/modules/attachments/files.py](../backend/app/modules/attachments/files.py)
Ключ хранения не зависит от имени файла	interactions/{id}/{attachment_id}, выход за каталог хранилища проверяется	[backend/app/core/storage.py](../backend/app/core/storage.py)

Транспорт и стенд

Мера	Как сделано	Где
TLS и перенаправление с 80 порта	Конфиг nginx стенда	[infra/nginx/nginx.conf](../infra/nginx/nginx.conf)
Заголовки безопасности	HSTS, nosniff, X-Frame-Options: DENY, Referrer-Policy, Permissions-Policy, CSP с frame-ancestors 'none'	там же
CORS только для известных адресов	Список из CORS_ORIGINS, по умолчанию пусто; * в продакшене запрещён	[backend/app/main.py](../backend/app/main.py)
Стенд не стартует с небезопасными настройками	При APP_ENV=production приложение отказывается запускаться без ключа шифрования, с неверным ключом, с паролем базы из локального профиля и с CORS_ORIGINS=*	[backend/app/core/config.py](../backend/app/core/config.py)
Секреты только в окружении	В коде и фикстурах паролей нет; в .env.example — заглушки	[backend/.env.example](../backend/.env.example)
Секреты не попадают в историю	gitleaks в CI по всей истории	[.github/workflows/backend.yml](../.github/workflows/backend.yml)
Уязвимые зависимости	pip-audit --strict в CI по requirements.lock — ровно те версии, что ставятся в образ	там же
Воспроизводимая сборка	Зависимости по backend/requirements.lock, базовый образ и образы стенда закреплены по digest, у рабочих процессов CI только чтение репозитория	[backend/Dockerfile](../backend/Dockerfile), [infra/docker-compose.prod.yml](../infra/docker-compose.prod.yml)

Резервное копирование

Мера	Как сделано	Где
Ежедневный дамп	Сервис backup: базы radar и keycloak, формат custom, хранение 14 дней на стенде	[infra/backup/backup.sh](../infra/backup/backup.sh)
Проверка восстановления	Оба дампа разворачиваются во временные базы и сверяются с рабочими	[infra/backup/restore-check.sh](../infra/backup/restore-check.sh)
Внешние копии	Шифрование открытым ключом gpg, отправка в стороннее S3; закрытый ключ вне сервера	[infra/backup/files.sh](../infra/backup/files.sh)
Ключи хранилища не видны в процессах	Конфиг mc во временном файле вместо аргументов	там же
Приложение не владеет схемой	Роль radar_app без DDL, история и аудит — только добавление; миграции под radar	[infra/postgres/roles.sql](../infra/postgres/roles.sql)
Отдельная база Keycloak	Роль keycloak со своим паролем, radar_app к ней не подключается	там же
Демо-пароли не уезжают на стенд	Подстановки ${DEMO_*_PASSWORD} в realm, на стенде обязательны	[infra/keycloak/realm-radar-vuzov.json](../infra/keycloak/realm-radar-vuzov.json)

Проверено на профиле стенда с чистого тома: roles.sql → migrate → демо-данные под radar_app → API и все задания воркера под ней же. Роль приложения получает отказ на UPDATE и TRUNCATE истории и аудита, на ALTER TABLE ... DISABLE TRIGGER, DROP TABLE, session_replication_role и подключение к базе Keycloak. Дампы обеих баз восстановлены (все таблицы, 351 взаимодействие, realm с тремя пользователями), внешние копии расшифрованы закрытым ключом байт в байт, отправка во внешний бакет прошла.

Нагрузочный тест и масштабирование

Проверяем НФТ из ТЗ: интерфейс отвечает быстрее секунды при полусотне одновременных пользователей (NFR-01, NFR-06) и выдерживает не меньше десяти параллельных отчётов (NFR-07).

Что нужно

• Стенд с данными v1: make db-up, make migrate, make seed-full в backend/.
• API в режиме разработки — нагрузка ходит с заголовком X-Dev-User, без Keycloak:
APP_ENV=local AUTH_MODE=dev make run.
• k6: brew install k6 на macOS,
пакет k6 из репозитория Grafana на Linux.

Сценарии

Файл	Что делает	Порог
[load/k6-main.js](../load/k6-main.js)	Пользователей столько, сколько сказано в VUS (по умолчанию 50): радар и матрица сигналов, список взаимодействий, карточка, заметка, рейтинг. Разгон 30 с, полка 2 мин, спад 30 с	p95 < 1000 мс, доля ошибок < 1 %
[load/k6-reports.js](../load/k6-reports.js)	10 пользователей заказывают отчёты в xlsx, json, pdf и xls одновременно	все задания доходят до done, p95 < 30 с

Запуск:

```
k6 run -e BASE_URL=http://127.0.0.1:8000 load/k6-main.js
k6 run -e BASE_URL=http://127.0.0.1:8000 -e VUS=300 load/k6-main.js
k6 run -e BASE_URL=http://127.0.0.1:8000 load/k6-reports.js
```

Результаты прогонов

Стенд: данные v1 на локальной машине (299 взаимодействий, 96 вузов, 10 548 метрик, 21 открытый сигнал), PostgreSQL и Redis в Docker, API и воркер — процессы на хосте. Числа с ноутбука, не с сервера: они показывают поведение системы, а не мощность железа стенда.

Рабочий день, 50 пользователей

Показатель	Значение	Порог
http_req_duration p95	758,86 мс	< 1000 мс ✓
медиана	162,77 мс	—
p90	512,75 мс	—
максимум	5,71 с	—
доля ошибок	0 % (0 из 18 330)	< 1 % ✓
пропускная способность	101,6 запроса/с, 3055 итераций за 3 минуты	—

Все проверки прошли: сигналы, матрица «КАМ × вид сигнала», список, карточка, заметка, рейтинг.

Повторный прогон 17.09

Те же данные v1 и тот же сценарий, но с кодом после доработок по ответам жюри: у списка, карточки и радара появились группа и клиент в выборке, у заметки — чтение порогов радара из настроек, у записи — отметки для исходящего обмена.

Показатель	Значение	Порог
http_req_duration p95	450,99 мс	< 1000 мс ✓
медиана	85,98 мс	—
p90	255,75 мс	—
максимум	2,67 с	—
доля ошибок	0 % (0 из 24 846)	< 1 % ✓
итераций	4141 за 3 минуты	—

Регрессии нет: доработки добавили по одному-два запроса на действие. Разница с первым прогоном в лучшую сторону объясняется прогретым кэшем PostgreSQL и тем, что в этот раз на ноутбуке не шла параллельная сборка, — это не ускорение кода.

Прогон 20.09

Те же данные v1 и тот же сценарий, но с кодом после статусов записи, ручных приоритетов, кэша, списков участников и сохранённых видов. Redis, PostgreSQL и MinIO в Docker, API и воркер — процессы на хосте.

Показатель	17.09	20.09	Порог
http_req_duration p95	450,99 мс	386,59 мс	< 1000 мс ✓
медиана	85,98 мс	181,76 мс	—
p90	255,75 мс	349,21 мс	—
максимум	2,67 с	908,25 мс	—
доля ошибок	0 %	0 % (0 из 21 048)	< 1 % ✓
итераций за 3 минуты	4141	3508	—

Хвост, на который мы смотрели в прошлый раз, исчез: максимум упал с 2,67 с до 0,91 с, а p95 — на 64 мс. Медиана при этом выросла: кэш снял пики, но сам по себе добавляет запрос в Redis на каждое чтение справочника. Порог с запасом, так что это честный размен.

Десять отчётов одновременно в том же прогоне: **10 из 10 доведены до done**, постановка задания p95 204 мс, полный цикл «заказ → готовый файл» от 1,21 до 4,32 с, ошибок нет (0 из 40 запросов).

Что даёт кэш. На тех же данных, с пустым Redis: рейтинг за период — 150 мс первый запрос и 6–9 мс следующие. Карточка взаимодействия отвечает одинаково быстро и с ETag, и без него (14 мс): тело ответа всё равно собирается, чтобы посчитать хэш, — но по сети уходит 0 байт вместо 7150. То есть кэш справочников и рейтинга экономит работу сервера, а ETag карточки — трафик и перерисовку в интерфейсе.

Десять отчётов одновременно

Показатель	Значение
заданий принято и доведено до done	10 из 10
время постановки задания	p95 229 мс
полный цикл «заказ → готовый файл»	от 1,4 с до 4,6 с
доля ошибок	0 % (0 из 40 запросов)

Форматы в прогоне чередовались: xlsx, json, pdf, xls.

Что нашлось

• Отчёты требуют живого воркера. Первый прогон показал 0 из 10 готовых: задания уходили в очередь arq, а воркер не был запущен. Это не дефект кода, но именно так это выглядит на стенде, если воркер упал, — поэтому в [docs/ADMIN_GUIDE.md](ADMIN_GUIDE.md) есть отдельный пункт про зависшие в queued отчёты.
• Дефект в самом сценарии. k6-reports.js проверял состояние отчёта сразу после создания, хотя отчёт строится в фоне. Исправлено: сценарий опрашивает состояние до done или failed.
• Хвост 5,71 с при 50 пользователях. Отдельные запросы на этих данных быстрые — рейтинг 131 мс, матрица сигналов 32 мс, список из 50 записей 52 мс, воронка 23 мс, — поэтому хвост даёт не тяжёлый запрос, а конкуренция: каждая заметка в сценарии пересчитывает сигналы взаимодействия в одной транзакции. Порог выдержан; для запаса пересчёт после заметки уводится в воркер (пункт 5 ниже).

Оптимизации не потребовались: оба порога НФТ выдержаны без изменений в коде.

300 пользователей и масштабирование

Жюри просило не только цифру по НФТ, но и рассказ о масштабировании на 300+ пользователей. Проверяли на том же ноутбуке: 8 ядер, 8 ГБ памяти, PostgreSQL, Redis и MinIO в Docker, k6 — там же. Данные v1, между прогонами заметки нагрузочного сценария удалялись и делался VACUUM ANALYZE, иначе каждый следующий прогон шёл по более толстой таблице.

Пользователей	Копий API	p95	Запросов/с	Ошибок
50	1	386,59 мс ✓	116,6	0 из 21 048
100	4	1,15 с ✗	138,5	0 из 24 996
300	1	2,19 с ✗	202,8	0 из 36 684
300	4	1,87 с ✗	226,4	0 из 40 872

Ошибок нет нигде. Система не отказывает и не теряет запросы — она замедляется: под нагрузкой растёт очередь, а не доля пятисотых. Это важнее самой цифры p95: отказ пришлось бы чинить,

замедление лечится железом.

Упирается не API, а база. На 100 пользователях PostgreSQL в контейнере занимает 420 % CPU (четыре ядра из восьми), Redis — 6 %. Поэтому четыре копии API вместо одной дают всего +12 % пропускной способности: копии ждут одну и ту же базу, которая делит процессор с ними и с самим k6. На пустом стенде каждый метод отвечает за 11–36 мс — ни одного медленного запроса нет, вся задержка под нагрузкой это конкуренция за ресурсы одной машины.

Как масштабировать на 300+

По убыванию отдачи:

1. Разнести по машинам. Сейчас база, приложение и генератор нагрузки делят восемь ядер. Отдельный сервер (или управляемый PostgreSQL) под базу — самый большой прирост из всех пунктов и единственный, который ничего не меняет в коде.
2. PgBouncer в режиме transaction. 300 пользователей на нескольких копиях API — это сотни соединений; пул на стороне базы дешевле, чем соединение на каждый процесс.
3. Копии API за nginx. Приложение без состояния: сессии в токене, файлы в S3, кэш и события в Redis — добавляется копиями без липких сессий (проверено: --workers 4 работает как есть).
4. Копии воркера. Очереди обмена и уведомлений захватываются арендой locked_until и SKIP LOCKED, поэтому вторая копия не отправит то же самое дважды.
5. Пересчёт сигналов после заметки — в воркер. Сейчас заметка пересчитывает радар записи в той же транзакции; при полусотне пользователей это незаметно, при трёхстах — самая частая запись в базу. Вынос в очередь убирает её из времени ответа.
6. Реплика для чтения. Отчёты, рейтинг и статистика читают много и не требуют последних миллисекунд — их можно увести на реплику, оставив мастеру работу КАМов.

Кэш готовых ответов (ADR-022) и файлы в S3 уже сделаны и работают на любой из этих конфигураций.

Замеры сделаны на машине разработчика: они показывают поведение системы и место узкого горлышка. Демо-стенд в Yandex Cloud (2 vCPU, 8 ГБ) рассчитан на показ, а для сотен пользователей масштабируется по шагам выше.

Куда смотреть, если порог не выдержан

- Списки взаимодействий и сигналов. Фильтры уже идут по индексированным полям (ix_interaction_owner_status, ix_interaction_stage_entered), но связанные сущности подтягиваются через joinedload: при росте страницы проверить план запроса и при необходимости перейти на selectinload.
- Рейтинг. Агрегаты по program_metric считаются на лету. Если именно они дают просадку — материализованное представление mv_program_metric_month и кэш в Redis по параметрам запроса, со сбросом после импорта и синхронизации.
- Отчёты. Строятся в воркере; узкое место обычно не в SQL, а в рендере файла. Помогает предел строк (50 000, уже есть) и запись xlsx в режиме write_only (уже есть).
- События SSE. Поток читает Redis с блокировкой; при большом числе подключений смотреть на число открытых соединений, а не на время запроса.

Диаграммы

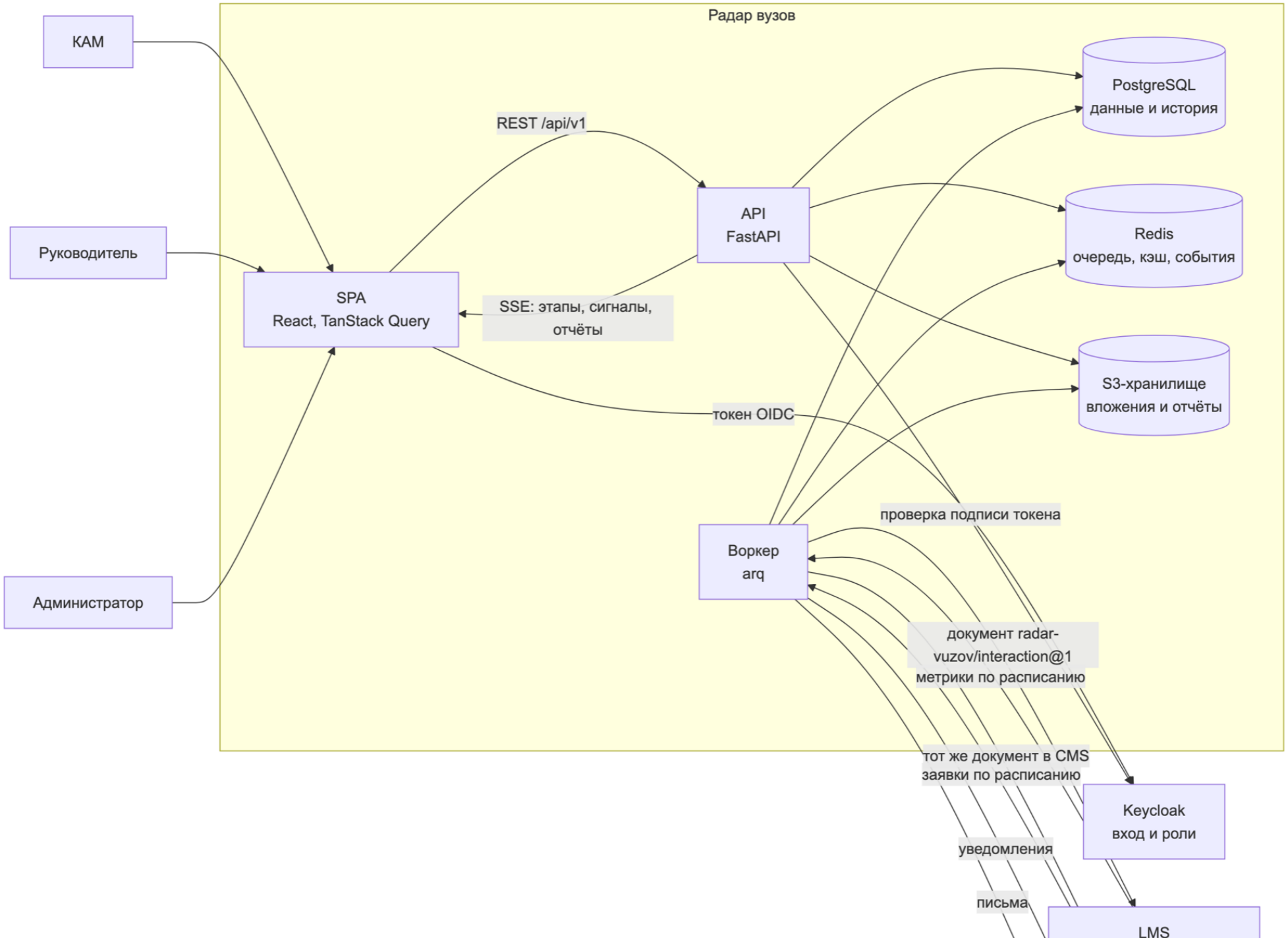
Три диаграммы, которые просило жюри: связи системы, программно-аппаратная архитектура и база данных. Исходники — Mermaid прямо в этом файле, GitHub рисует их сам. Полная модель в ArchiMate лежит рядом: model.xml, открывается в Archi (File → Import → Model from Open Exchange File).

Пояснения к решениям — в [ARCHITECTURE.md](../../ARCHITECTURE.md), расчёты — в [DATA_PROCESSING.md](../DATA_PROCESSING.md).

Те же диаграммы отрисованы в картинки — [images/](images): они идут в сопроводительный PDF, где Mermaid не нарисуешь. Пересобрать после правки исходников:
make docs-pdf вставит свежие файлы, а сами картинки снимаются headless-браузером: страница с диаграммой mermaid 11 открывается в Chrome, у отрисованного SVG берутся настоящие ширина и высота, и снимок делается ровно под них (--headless --window-size=W,H --screenshot).

1. Диаграмма связей

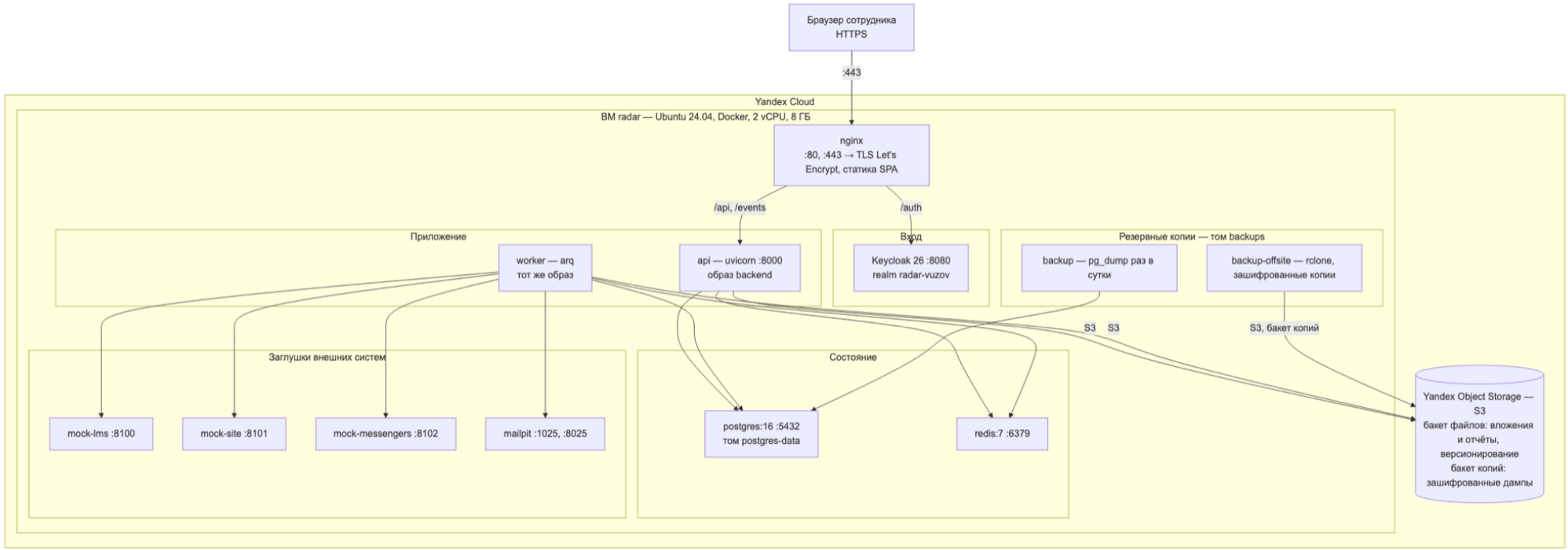
Кто с кем разговаривает и по какому поводу. Стрелка — направление вызова, подпись — что передаётся.


Радар вузов
КАМ
Руководитель
Администратор
SPA
React, TanStack Query
REST /api/v1
SSE: этапы, сигналы, отчёты
токен OIDC
API
FastAPI
Воркер
arq
PostgreSQL
данные и история
Redis
очередь, кэш, события
S3-хранилище
вложения и отчёты
проверка подписи токена
документ radar-vuzov/interaction@1
метрики по расписанию
тот же документ в CMS
заявки по расписанию
Keycloak
вход и роли
уведомления
письма
LMS

Что здесь важно. Обмен двусторонний: входящий — по расписанию, исходящий — очередью integration_outbox, чтобы отказ получателя не ронял работу КАМа (ADR-018). Уведомления уходят из воркера, а не из запроса: медленный мессенджер не должен задерживать переход по этапу (ADR-016). Права проверяет только API — SPA лишь прячет недоступное.

2. Программно-аппаратная архитектура

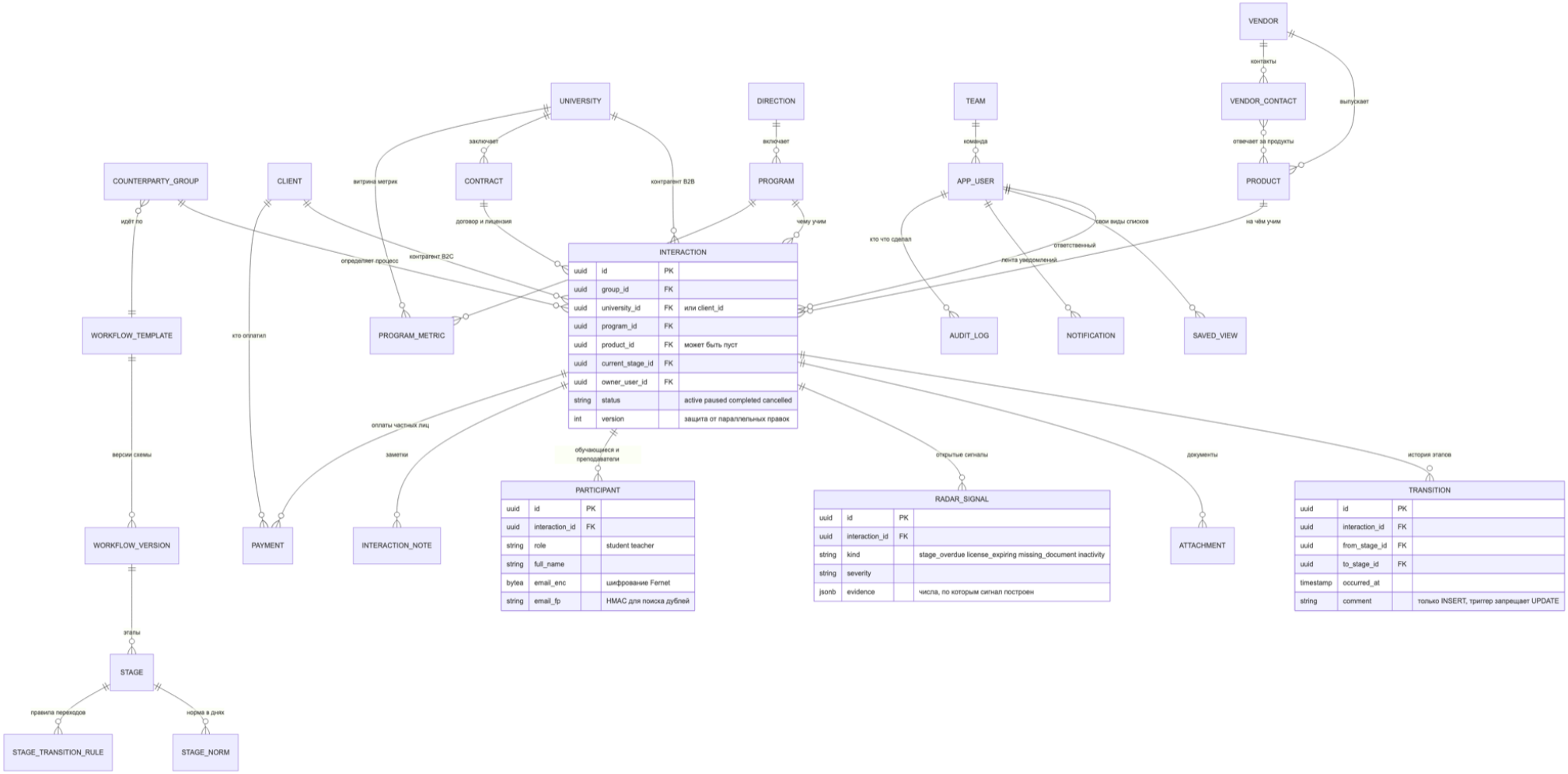
Стенд — одна виртуальная машина в Yandex Cloud: всё поднимается docker compose из [infra/docker-compose.prod.yml](../../infra/docker-compose.prod.yml) с дополнением [docker-compose.yandex.yml](../../infra/docker-compose.yandex.yml). Файлы лежат не на машине, а в Yandex Object Storage — облачном S3-хранилище. Наружу открыты только 80 и 443 (и 22 для администратора). В закрытом контуре заказчика вместо Object Storage ставится MinIO или другое S3-хранилище — код тот же, меняются адрес и ключи.


Браузер сотрудника
HTTPS
:443
Yandex Cloud
ВМ radar — Ubuntu 24.04, Docker, 2 vCPU, 8 ГБ
nginx
:80, :443 → TLS Let's Encrypt, статика SPA
/api, /events
/auth
Приложение
worker — arq
тот же образ
api — uvicorn :8000
образ backend
Вход
Keycloak 26 :8080
realm radar-vuzov
Резервные копии — том backups
backup — pg_dump раз в сутки
backup-offsite — rclone, зашифрованные копии
S3
S3
S3, бакет копий
Заглушки внешних систем
mock-lms :8100
mock-site :8101
mock-messengers :8102
mailpit :1025, :8025
Состояние
postgres:16 :5432
том postgres-data
redis:7 :6379
Yandex Object Storage — S3
бакет файлов: вложения и отчёты, версионирование
бакет копий: зашифрованные дампы

Масштабирование. Узкое место — API: он без состояния, поэтому добавляется копиями за тем же nginx. Воркер тоже масштабируется копиями: очереди обмена и уведомлений захватываются арендой (locked_until) и SKIP LOCKED, поэтому две копии не отправят одно и то же дважды. Тяжёлое чтение (рейтинг, справочники) снимается кэшем в Redis (ADR-022), файлы лежат в S3 и не грузят базу. Подробности и замеры — в [LOAD_TEST.md](../LOAD_TEST.md).

3. База данных

Главные таблицы и связи между ними. Полная модель со всеми полями — в разделе «Модель данных» [ARCHITECTURE.md](../../ARCHITECTURE.md); служебные таблицы импорта, интеграций и уведомлений здесь опущены, чтобы схема читалась.


VENDOR
контакты
VENDOR_CONTACT
выпускает
отвечает за продукты
PRODUCT
на чём учим
UNIVERSITY
заключает
CONTRACT
договор и лицензия
витрина метрик
контрагент B2B
DIRECTION
включает
PROGRAM
чему учим
TEAM
команда
APP_USER
свои виды списков
ответственный
лента уведомлений
кто что сделал
COUNTERPARTY_GROUP
идёт по
CLIENT
кто оплатил
определяет процесс
контрагент B2C
INTERACTION
uuid id PK
uuid group_id FK
uuid university_id FK или client_id
uuid program_id FK
uuid product_id FK может быть пуст
uuid current_stage_id FK
uuid owner_user_id FK
string status active paused completed cancelled
int version защита от параллельных правок
PROGRAM_METRIC
AUDIT_LOG
NOTIFICATION
SAVED_VIEW
WORKFLOW_TEMPLATE
версии схемы
WORKFLOW_VERSION
этапы
STAGE
правила переходов
STAGE_TRANSITION_RULE
норма в днях
STAGE_NORM
оплаты частных лиц
PAYMENT
заметки
INTERACTION_NOTE
обучающиеся и преподаватели
PARTICIPANT
uuid id PK
uuid interaction_id FK
string role student teacher
string full_name
bytea email_enc шифрование Fernet
string email_fp HMAC для поиска дублей
открытые сигналы
RADAR_SIGNAL
uuid id PK
uuid interaction_id FK
string kind stage_overdue license_expiring missing_document inactivity
string severity
jsonb evidence числа, по которым сигнал построен
документы
ATTACHMENT
история этапов
TRANSITION
uuid id PK
uuid interaction_id FK
uuid from_stage_id FK
uuid to_stage_id FK
timestamp occurred_at
string comment только INSERT, триггер запрещает UPDATE

Что важно в схеме. Контрагент у записи ровно один: либо вуз, либо клиент — условие проверяет база (ADR-015). Частичные уникальные индексы не дают завести вторую активную запись по той же связке «контрагент — программа — продукт». История переходов и журнал аудита только дописываются: это обеспечивает триггер, а не договорённость.

Соответствие техническому заданию

Каждое требование кейса №6 — формулировка из ТЗ, где оно выполнено и чем это проверено.

Итог да — требование выполнено и проверено способом из последней колонки.

Функциональные требования

ID	Требование ТЗ	Итог	Как сделано	Чем проверено
SR-01	Каталоги (ИТ-продукты, направления, вузы, ответственные) в БД, обновление через интерфейс загрузкой xls/xlsx по маппингу полей	да	Каталоги в PostgreSQL; загрузка выгрузки заказчика (xls, xlsx, csv) с профилями соответствия колонок и предпросмотром — app/modules/imports/; справочники файлом в админке (JSON, CSV, XLSX, XLS), включая контакты вендоров из таблицы кейсодержателя — app/modules/admin/catalog_io.py	tests/test_imports.py, tests/test_catalog_files.py, tests/test_encodings.py; повторная загрузка того же файла даёт «без изменений»
SR-02	Отчёты по всем взаимодействиям с вузами за выбранный период	да	Отчёт за период по всем взаимодействиям с фильтрами и выбором колонок — app/modules/reports/	tests/test_reports.py; под нагрузкой k6 10 отчётов одновременно доходят до done (LOAD_TEST.md)
FR-01	Фильтрация по периоду, вузам, направлениям, продуктам, ответственным; статистика в диаграммах (png, pdf)	да	Фильтры по периоду, вузу, направлению, продукту, ответственному, этапу и состоянию — во всех списках и в отчёте; статистика: воронка, длительности этапов, распределение — app/modules/analytics/stats.py; диаграммы в PDF рисует сервер (GET /analytics/stats/report), PNG отдаёт ECharts на клиенте	tests/test_analytics.py, tests/test_reports.py; PDF со статистикой отрисован в картинку и просмотрен глазами
FR-02	Визуализация пути взаимодействия: переходы, переименование статусов, комментарии при переходе	да	Путь взаимодействия: этапы, правила переходов с обязательными комментарием и документом, возвраты на шаг назад, переименование этапов администратором, история только на дозапись — app/modules/workflow/, app/modules/interactions/	tests/test_interactions.py, tests/test_workflow_editor.py; триггер БД отклоняет UPDATE перехода
FR-03	Файлы в статусах: png, jpeg, pdf, zip, gzip, rar, doc, docx, xls, xlsx	да	Разрешены png, jpeg, pdf, zip, gzip, rar, doc, docx, xls, xlsx; тип проверяется по расширению и сигнатуре файла, размер ограничен — app/modules/attachments/files.py	tests/test_attachments.py: подменённое расширение отклоняется с FILE_TYPE_NOT_ALLOWED
FR-04	Отчёты за период в xls, xlsx, pdf по выбранным колонкам	да	Отчёт за период в xlsx, xls, csv, pdf и json по выбранным колонкам — app/modules/reports/renderers.py	tests/test_reports.py: у каждого формата своя сигнатура файла; tests/test_encodings.py: выгрузка читается обратно нашим же импортом
FR-05	Получение JSON по API из LMS и сайта с добавлением в существующий или новый workflow	да	Приём JSON из LMS и с сайта по расписанию, сопоставление заявок с существующими записями и заведение новых; обратная выгрузка документом radar-vuzov/interaction@1 — app/modules/integrations/, app/modules/exchange/	tests/test_integrations.py, tests/test_exchange.py; на стенде обмен прогнан с заглушками LMS и сайта по HTTP
FR-06	Креатив и ИИ внутри (у нас — радар и рейтинг)	да	Радар четырёх сигналов и рейтинг востребованности с объяснимым вкладом каждой метрики, без внешних LLM (ADR-006) — app/modules/radar/, app/modules/analytics/rating.py	tests/test_radar_rules.py, tests/test_signals.py, tests/test_analytics.py: сумма вкладов равна баллу
FR-07	Соответствие 152-ФЗ и приказу ФСТЭК №117	да	Меры и их место в коде — SECURITY.md: шифрование ПДн, аудит просмотров, области видимости, журнал только на дозапись, TLS и заголовки в nginx	Тест «чужая запись недоступна» на каждом эндпоинте с данными; tests/test_auth.py, tests/test_admin.py

ID	Требование ТЗ	Итог	Как сделано	Чем проверено
FR-08	Отчёт выдаётся файлом xls, xlsx или pdf	да	Тот же отчёт выдаётся файлом с исходным именем и типом — GET /reports/{id}/file	tests/test_reports.py
FR-09	Создание и изменение workflow	да	Создание и изменение процесса: черновик, предпросмотр публикации, перенос открытых записей, группы контрагентов со своим процессом — app/modules/workflow/editor.py	tests/test_workflow_editor.py, tests/test_groups.py: удалённый этап не останавливает работу
FR-10	Авторизация через Keycloak	да	Вход через Keycloak (OIDC, код с PKCE), проверка подписи и аудитории токена — app/core/security.py; realm с политиками — infra/keycloak/	Живая проверка 17.09: токен kam.demo, полученный по коду с PKCE, принят /api/v1/me
FR-11	Ролевая модель: пользователь, руководитель, администратор	да	Роли kam, manager, admin и области видимости, плюс точечные правила доступа администратора — app/core/scope.py, app/modules/admin/	tests/test_auth.py, tests/test_admin.py, tests/test_bulk_and_owner.py
FR-12	Веб-интерфейс	да	Веб-интерфейс на React 19 и TypeScript — frontend/: радар, списки и карточка записи, процесс, отчёты, рейтинг и статистика, импорт, клиенты, оплаты, администрирование, справка; вход через Keycloak	npm test и npm run build в CI (.github/workflows/frontend.yml); интерфейс работает на стенде
FR-13	Кэш действий пользователя	да	Кэш готовых ответов справочников и рейтинга в Redis + условные запросы (ETag, 304) у карточки, списков и файлов — app/core/cache.py, app/core/http_cache.py (ADR-022)	tests/test_cache.py; на стенде: рейтинг 150 мс против 6–9 мс, карточка по If-None-Match — 0 байт вместо 7150

Нефункциональные требования

ID	Требование ТЗ	Итог	Как сделано	Чем проверено
NFR-01	Отклик интерфейса не больше 1 секунды	да	Индексы под фильтры списков, кэш тяжёлых ответов, тяжёлое — в воркер	k6 на данных v1: p95 386,59 мс при 50 пользователях, ноль ошибок (LOAD_TEST.md)
NFR-02	Без перезагрузки страницы при изменениях	да	Поток событий SSE GET /api/v1/events с Last-Event-ID: переходы, сигналы, отчёты, уведомления, сообщения	tests/test_events.py: событие приходит только тому, кому запись видна
NFR-03	Коды ошибок	да	Каталог из 18 кодов ошибок (17 возвращаются по HTTP, INTEGRATION_UNAVAILABLE — в журнале запусков обмена), ответы application/problem+json с trace_id — app/core/errors.py, каталог в [ARCHITECTURE.md](../ARCHITECTURE.md)	tests/test_system.py и тесты каждого модуля проверяют коды
NFR-04	Интуитивный интерфейс	да	Работа начинается с радара: сигнал ведёт в карточку, где показаны только допустимые переходы и их требования; диаграммы открывают списки с нужным фильтром; встроенная справка по роли	Сценарии КАМа, руководителя и администратора пройдены на стенде
NFR-05	Документация пользователя и администратора со скриншотами, встроенная в платформу	да	Руководство пользователя встроено в продукт: разделы по ролям GET /api/v1/help, скриншоты работающих экранов — GET /api/v1/help/images/{name}; тот же текст со снимками собирается в USER_GUIDE.md и в общий PDF (ADR-025); руководство администратора — ADMIN_GUIDE.md	tests/test_help.py: картинка отдаётся, чужое имя и попытка выйти из каталога — 404, без входа — 401; документ и справка показывают одни и те же снимки
NFR-06	50 параллельных пользователей	да	50 параллельных пользователей	k6: 21 048 запросов, ноль ошибок, p95 386,59 мс
NFR-07	Не меньше 10 параллельных отчётов	да	Очередь отчётов на arq с прогрессом	k6: 10 отчётов одновременно, все дошли до done, постановка p95 204 мс

Решение и оформление

ID	Требование ТЗ	Итог	Как сделано	Чем проверено
SOL-01	Стек: Python, React, PostgreSQL	да	Python 3.12 + FastAPI, React 19, PostgreSQL 16	backend/pyproject.toml, frontend/package.json
SOL-02	Swagger UI для всех методов и перечень библиотек	да	Swagger UI /api/docs для всех методов, контракт contracts/openapi.yaml (107 путей, 122 операции), перечень библиотек с точными версиями — [backend/requirements.lock](../backend/requirements.lock) и [frontend/package.json](../frontend/package.json)	tests/test_contract.py: контракт совпадает со схемой приложения, расхождение ловит CI
SOL-03	Отдельный независимый сервис	да	Отдельный сервис: API и воркер в своих контейнерах, внешние системы — за интерфейсами клиентов	infra/docker-compose.yml, infra/docker-compose.prod.yml
SOL-04	Результирующий JSON и Docker-контейнеры	да	JSON во всех ответах и в обмене; Docker-образ бэкенда и профили compose	Образ собирается с requirements.lock; в CI тесты идут против контейнера PostgreSQL, Redis и S3 в них подменены реализациями в памяти (MemoryCache, каталог на диске). С Redis и S3 приложение работает на стенде (Object Storage) и в локальном compose (MinIO)
SOL-05	Комментарии в сложных местах кода	да	Комментарии объясняют «почему» в неочевидных местах: аренда очередей, отпечаток почты, поколения кэша	Независимые ревью кода отдельно отметили это
SOL-06	Сопроводительная документация: методы, ограничения, сборка и установка, архитектура в Archi	да	Архитектура и решения — [ARCHITECTURE.md](../ARCHITECTURE.md), методы обработки — DATA_PROCESSING.md, сборка и установка — DEPLOY.md, модель Archi — architecture/model.xml, диаграммы — architecture/DIAGRAMS.md	XML модели разбирается, висячих ссылок нет; диаграммы отрисованы mermaid без ошибок
SOL-07	Открытый необфусцированный код	да	Код открытый и необфусцированный: исходники бэкенда и фронтенда, миграции, тесты и контракт API в репозитории, сборка — из исходников	Образы собираются из репозитория в CI (.github/workflows/backend.yml, frontend.yml)
SOL-08	Работа на Linux	да	Образы на python:3.12-slim, CI на Ubuntu	CI прогоняет тесты на Ubuntu; стенд — docker compose на Linux
SOL-09	149-ФЗ и 152-ФЗ	да	Синтетические данные, шифрование ПДн, аудит, области видимости — SECURITY.md	Тесты приватности и аудита; реальных персональных данных в репозитории нет
UX-01	Удобство на desktop и mobile, строгий стиль, понятный путь пользователя	да	Светлый интерфейс с тёмным меню в цветах брендбука, адаптивная вёрстка; на телефоне все разделы — через боковое меню	Интерфейс проверен в браузере на ширине 375–390 px

Сдача

ID	Требование ТЗ	Итог	Как сделано	Чем проверено
SUB-01	Ссылка на репозиторий	да	https://github.com/tame2tame/hackathon_2026	—
SUB-03	Ссылка на работающий прототип с доступом	да	Работающий прототип по ссылке: https://158-160-195-122.sslip.io — Yandex Cloud, TLS Let's Encrypt, файлы в Object Storage, демо-данные v1 (DEPLOY.md)	infra/smoke.sh по адресу стенда 16.10: все восемь проверок прошли (API, база, хранилище, Swagger, Keycloak, закрытый доступ без входа)

ID	Требование ТЗ	Итог	Как сделано	Чем проверено
SUB-04	Сопроводительная документация (.doc/.pdf)	да	Сопроводительная документация одним файлом — РадарВузов-документация.pdf: архитектура, методы обработки, оба руководства, установка, меры защиты, нагрузка и этот разбор. Собирается командой make docs-pdf из тех же Markdown-исходников, без pandoc и браузера	tests/test_docs_pdf.py: сборка воспроизводима, пропавший исходник останавливает её